

Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria de Ingeniería Campus Tlaxcala

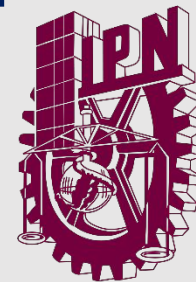
Ingeniería en Inteligencia Artificial

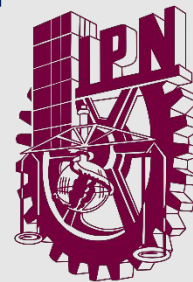
Algoritmos y estructura de datos

Ricardo Ramos Aguilar

Primavera 2022



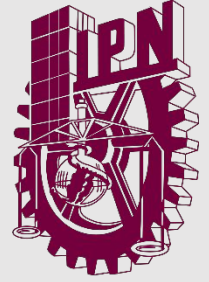




Data structures!



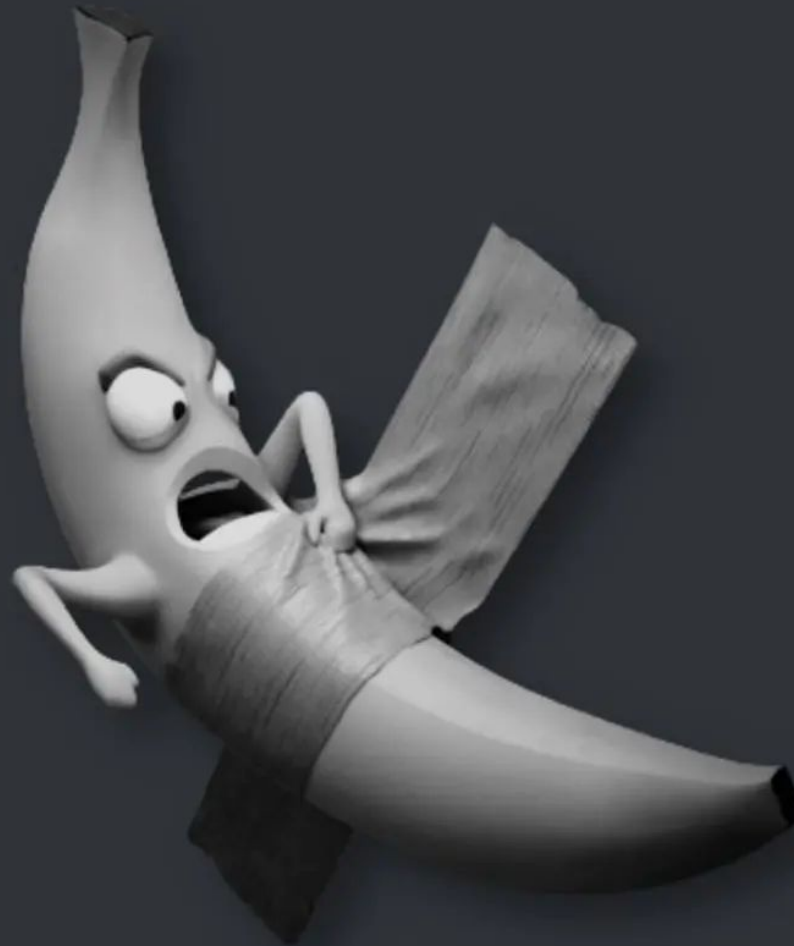
meme-arsenal.ru

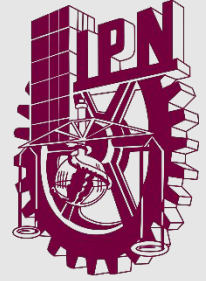


WHY ARE THEY IMPORTANT DATA STRUCTURES

 @codechips

Art Credit :
@the.rendercode



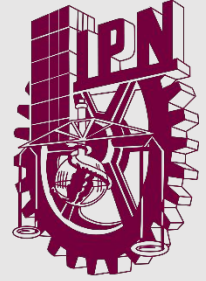


What is a Data Structure ?

DS is a way to store and organize data so that it can be used efficiently

lets use DS now !





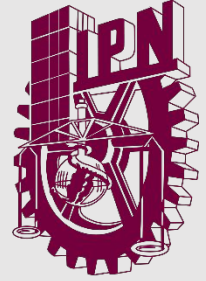
Knowing what data structure to use is necessary to



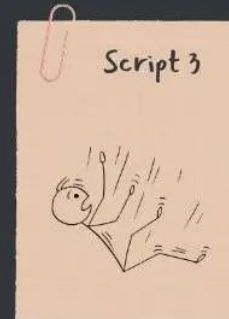
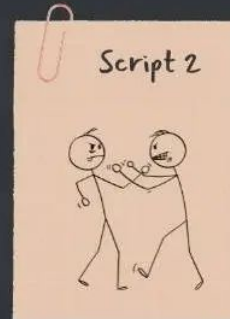
to allow for reasonable
processor speeds

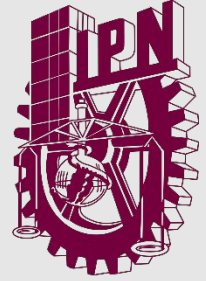
make and support data
searchable, sortable &
mergeable

to handle high
levels of requests

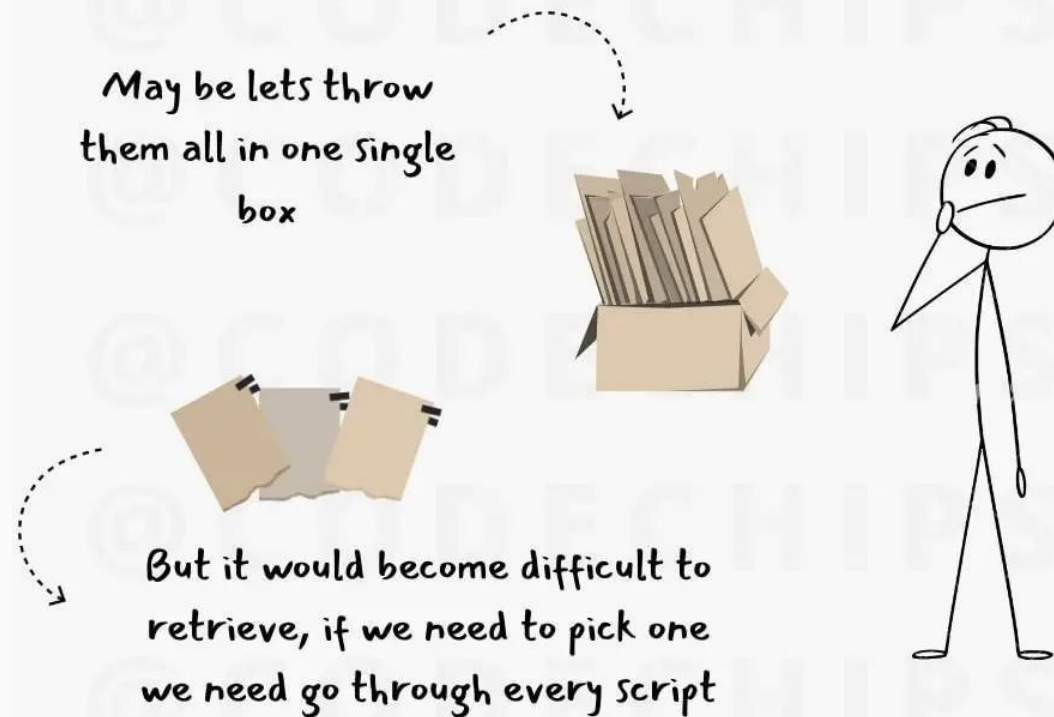


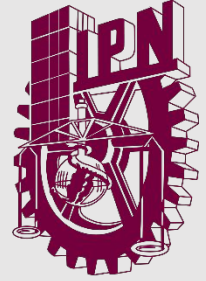
Lets say you have 3 story
scripts and you need to store it safely





You have number of possible ways to store it
But it should be **efficient** to access and
manage the scripts later when needed



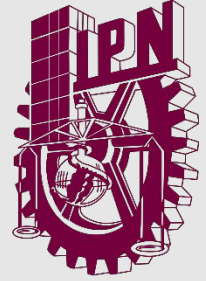


These can be stored in an Array

Think of it as a series of boxes



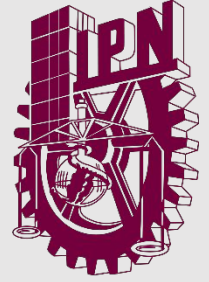
Now when we need to get a particular script we can quickly access it by its **index** i.e. Box number



What if all 3 scripts are interlinked stories and you need to have connectivity between them

Now **Linked list** can be used to store them

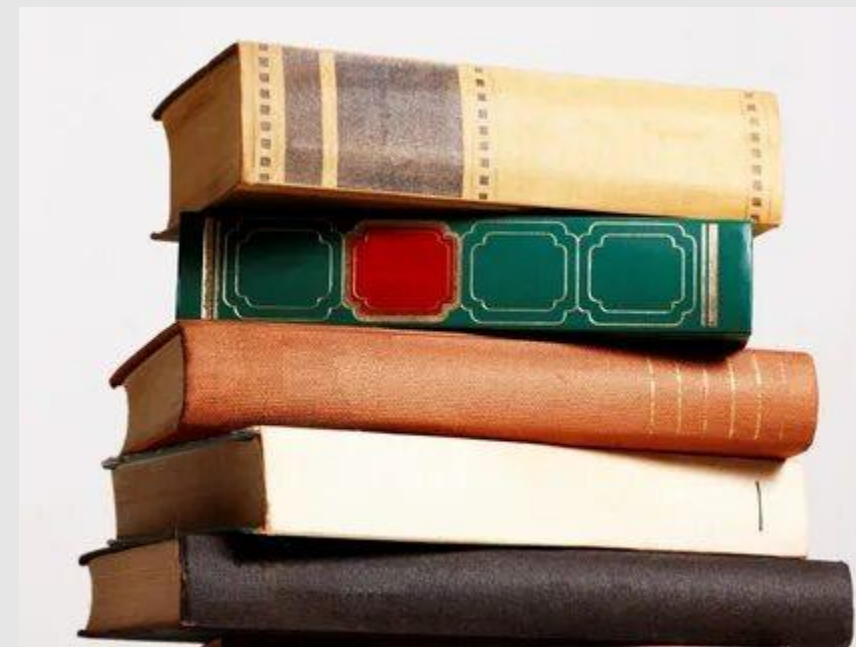
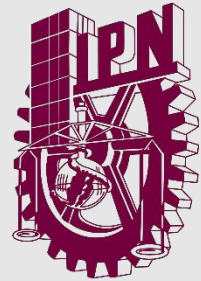


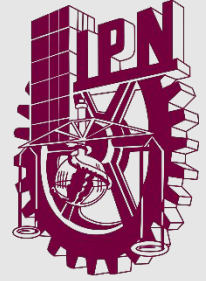


Understanding the complexity, the algorithm efficiency, best tool for the job separates the average from the above average programmer



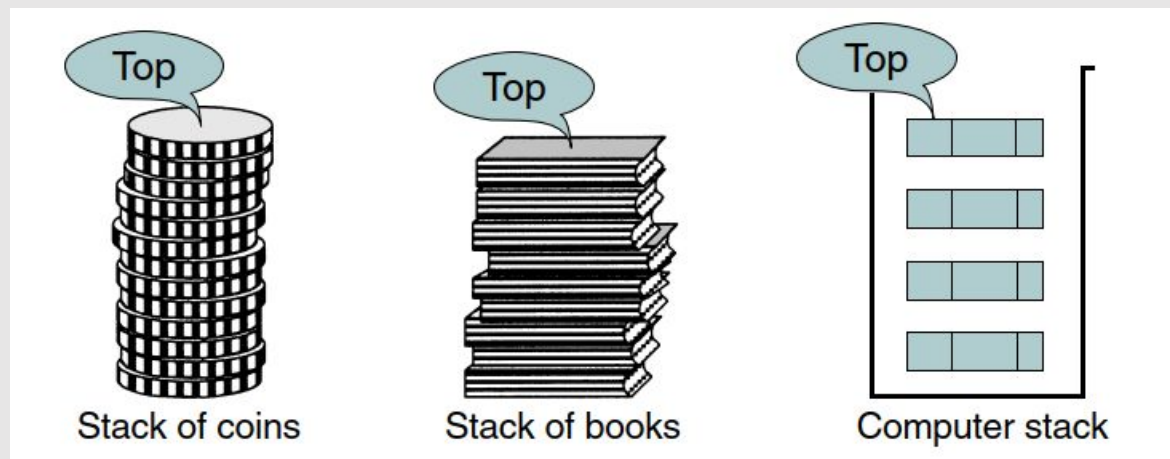
Stacks

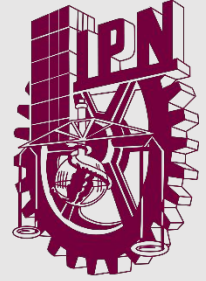




Stacks

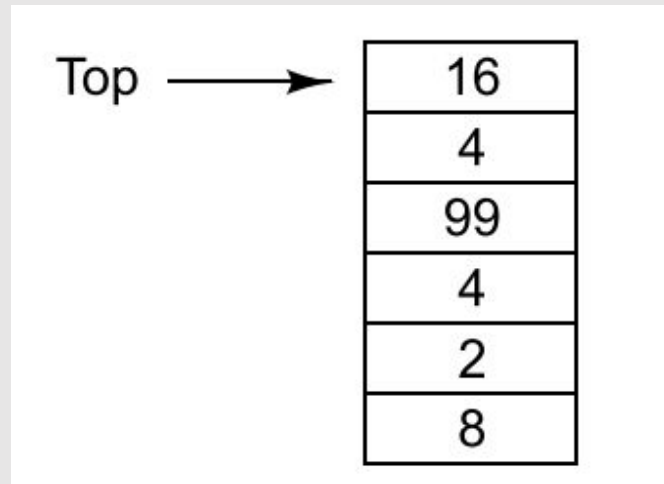
Stack is a linear data structure in which items are added or removed only at one end, called top of the stack. Thus, there is no way to add or delete elements anywhere else in the stack. A stack is based on Last-In-First-Out (LIFO) principle that means the data item that is inserted last into the stack is the first one to be removed from the stack.



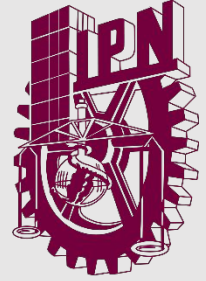


Stacks

Stacks appear as a group of elements stored at contiguous locations in memory. Each successive insert or delete operation adds or removes an item from the group. The top location of the stack or the point of addition or deletion is maintained by a pointer called top.



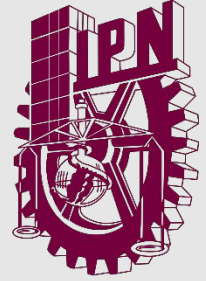
The logical representation of stacks showing stack elements stored at contiguous memory location might be true in case of their array implementation but the same might not be true in case of their linked implementation.



Arrays vs. Stacks

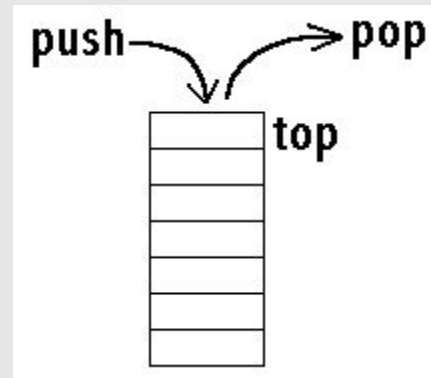
- While both arrays and stacks may look to be similar in their logical representation, they are different in several aspects.

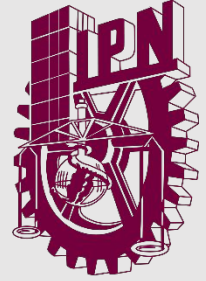
Arrays	Stacks
Arrays provide the flexibility of adding or removing data elements anywhere in the list, i.e., at the beginning, end or anywhere in the middle. While this flexibility may seem to be a boon in certain situations, the same may not be true in situations where frequent insertions or deletions are required. This is because; each insertion or deletion in arrays requires the adjoining elements to be shifted to new locations, which is an overhead.	Stacks restrict the insertion or deletion of elements to only one place in the list i.e. the top of the stack. Thus, there are no associated overheads of shifting other elements to new locations.
By using arrays, a programmer can realize common scenarios where grouping of records is required, for example inventory management, employee records management, etc.	Stacks find their usage as vital in solutions to advanced systems problems such as recursion control, expression evaluation, etc.



Basic Stack Operations

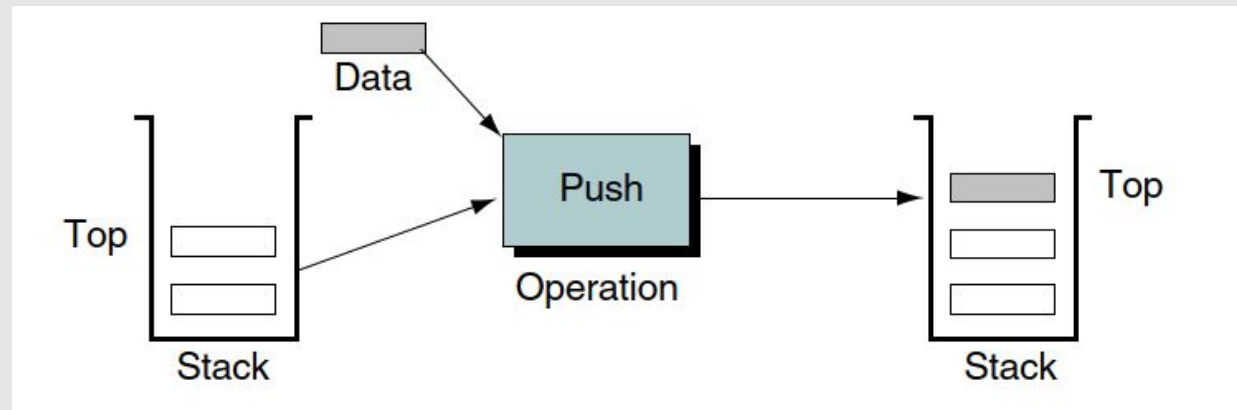
- The three basic stack operations are **push**, **pop**, and **stack top**. Push is used to insert data into the stack. Pop removes data from a stack and returns the data to the calling module. Stack top returns the data at the top of the stack without deleting the data from the stack.

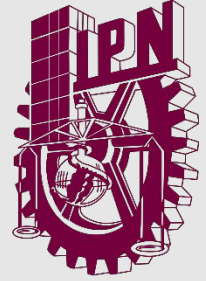




Push

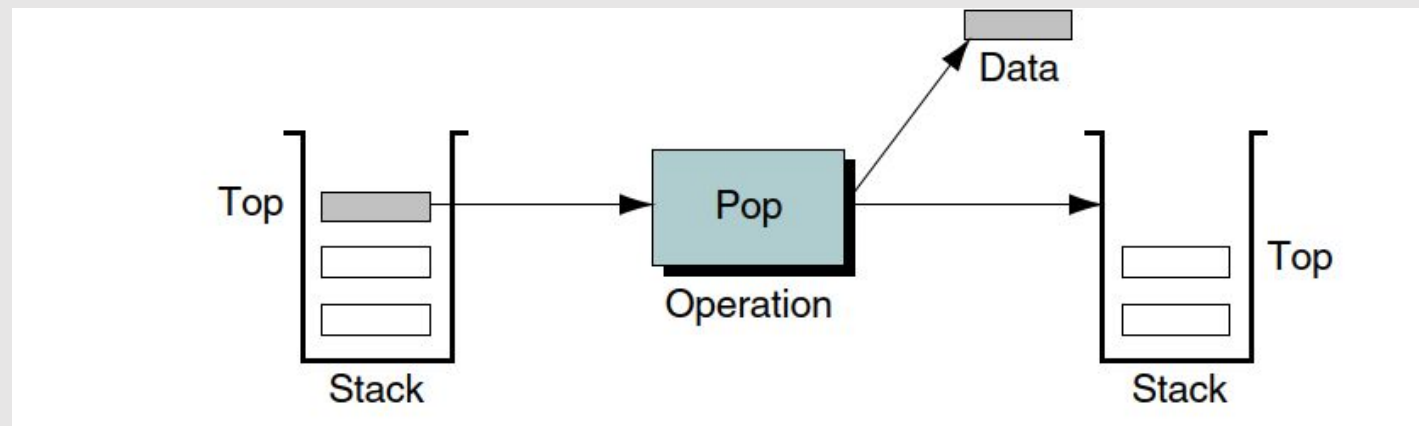
- Push adds an item at the top of the stack. After the push, the new item becomes the top. The only potential problem with this simple operation is that we must ensure that there is room for the new item. If there is not enough room, the stack is in an overflow state and the item cannot be added.



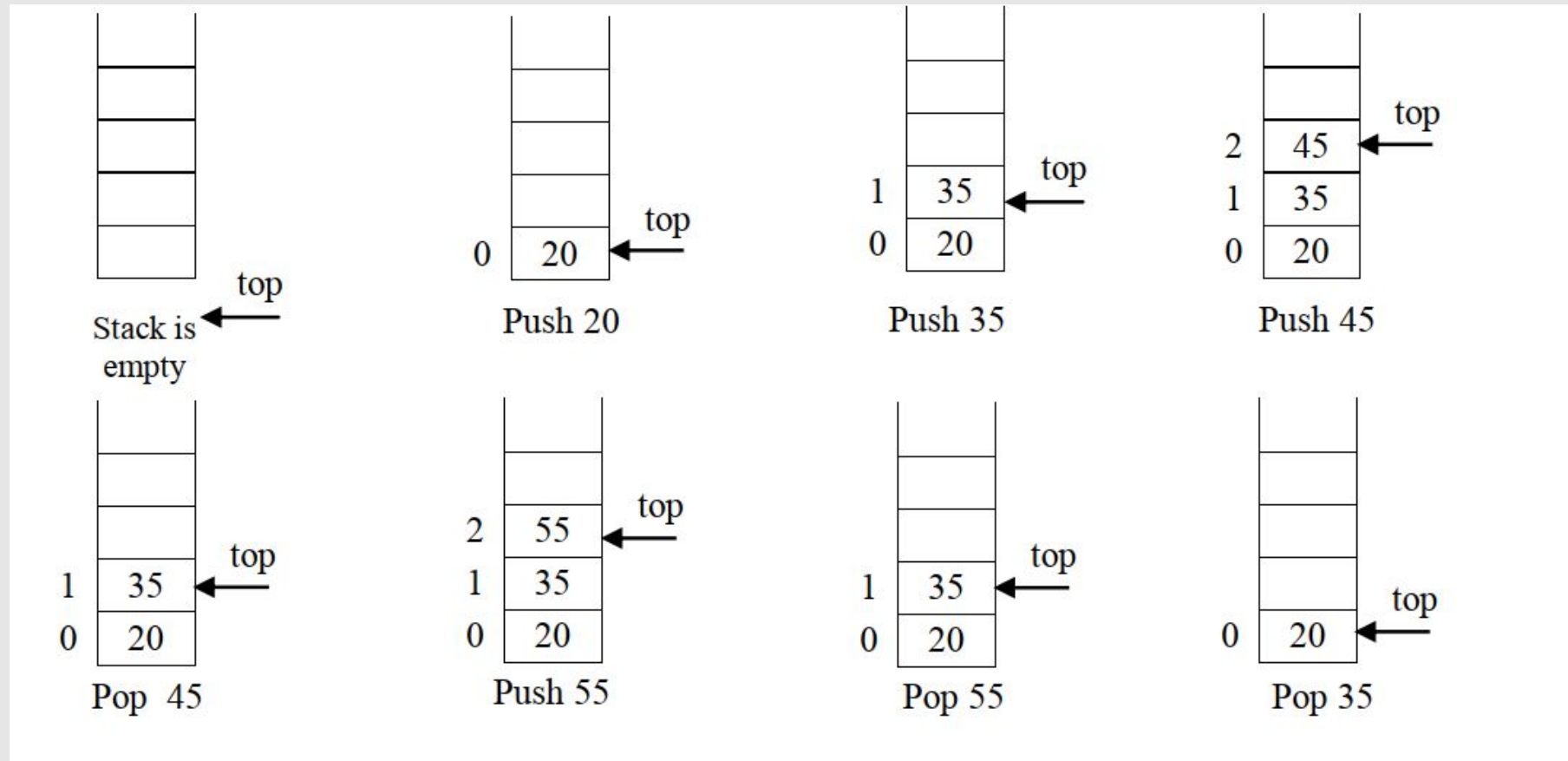
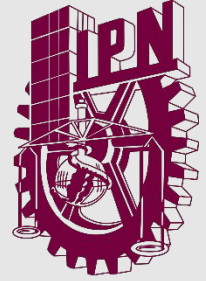


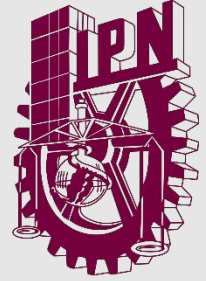
Pop

When we pop a stack, we remove the item at the top of the stack and return it to the user. Because we have removed the top item, the next older item in the stack becomes the top. When the last item in the stack is deleted, the stack must be set to its empty state. If pop is called when the stack is empty, it is in an underflow state.



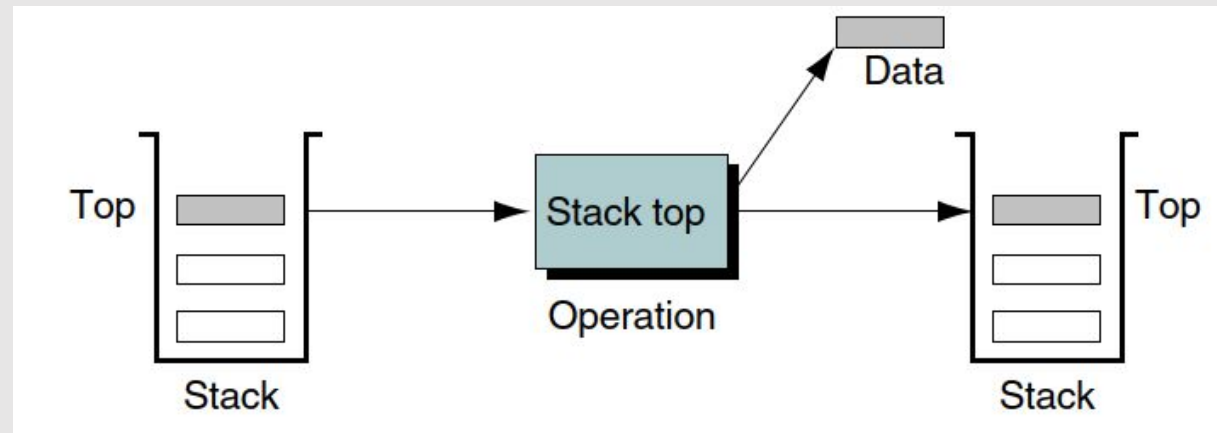
Example



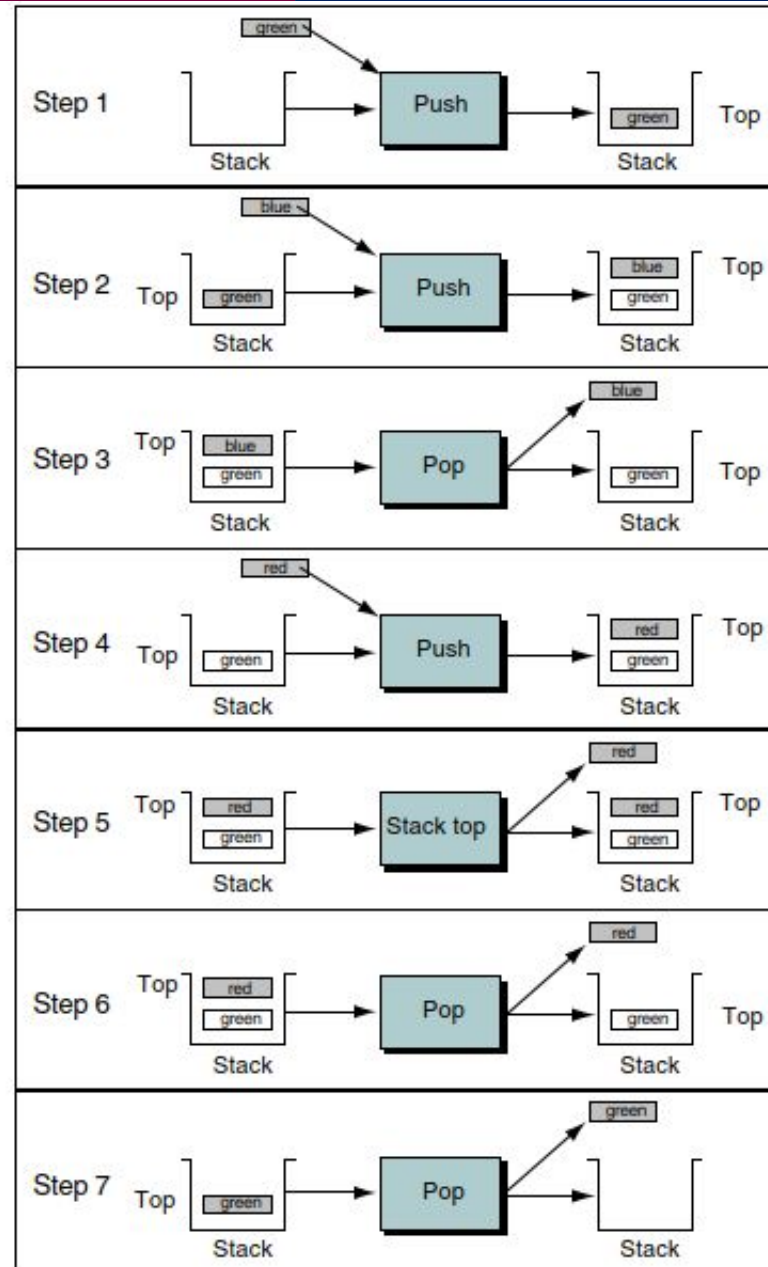
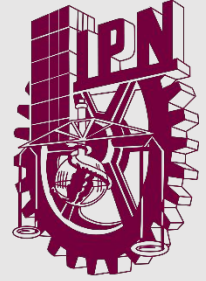


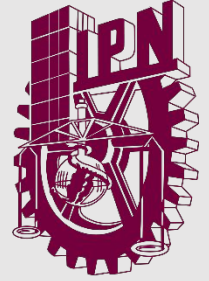
Stack Top

The third stack operation is stack top. Stack top copies the item at the top of the stack; that is, it returns the data in the top element to the user but does not delete it. You might think of this operation as reading the stack top. Stack top can also result in underflow if the stack is empty.



Example





Stack Specification

A stack is either *empty* or it consists of two parts:

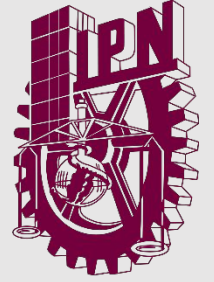
- a top element
- a stack (the remaining elements).

The elements in a stack may be of any type, but all the elements in a given stack must be the same type.

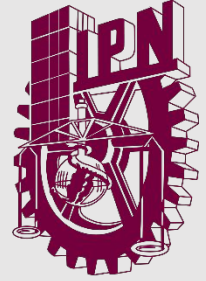
The operations on stacks are:

- Push
- Pop
- Top
-

HW



- Estudiar apuntadores y estructuras



STACK IMPLEMENTATION

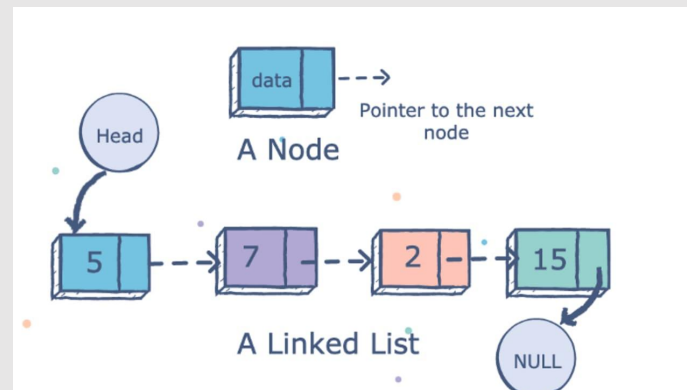
Stack can be implemented in two ways:

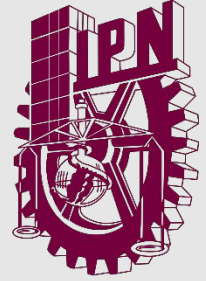
1. Static implementation

1. Static implementation is done with the help of Array

2. Dynamic implementation

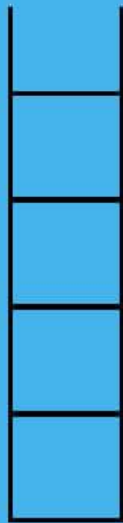
1. Is also called linked list representation, and uses pointer to implement the stack type of Data structure.





Considerations

Stack overflow



Stack underflow

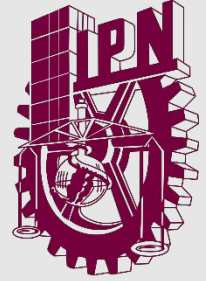
Fixed stack implementation

No overflow



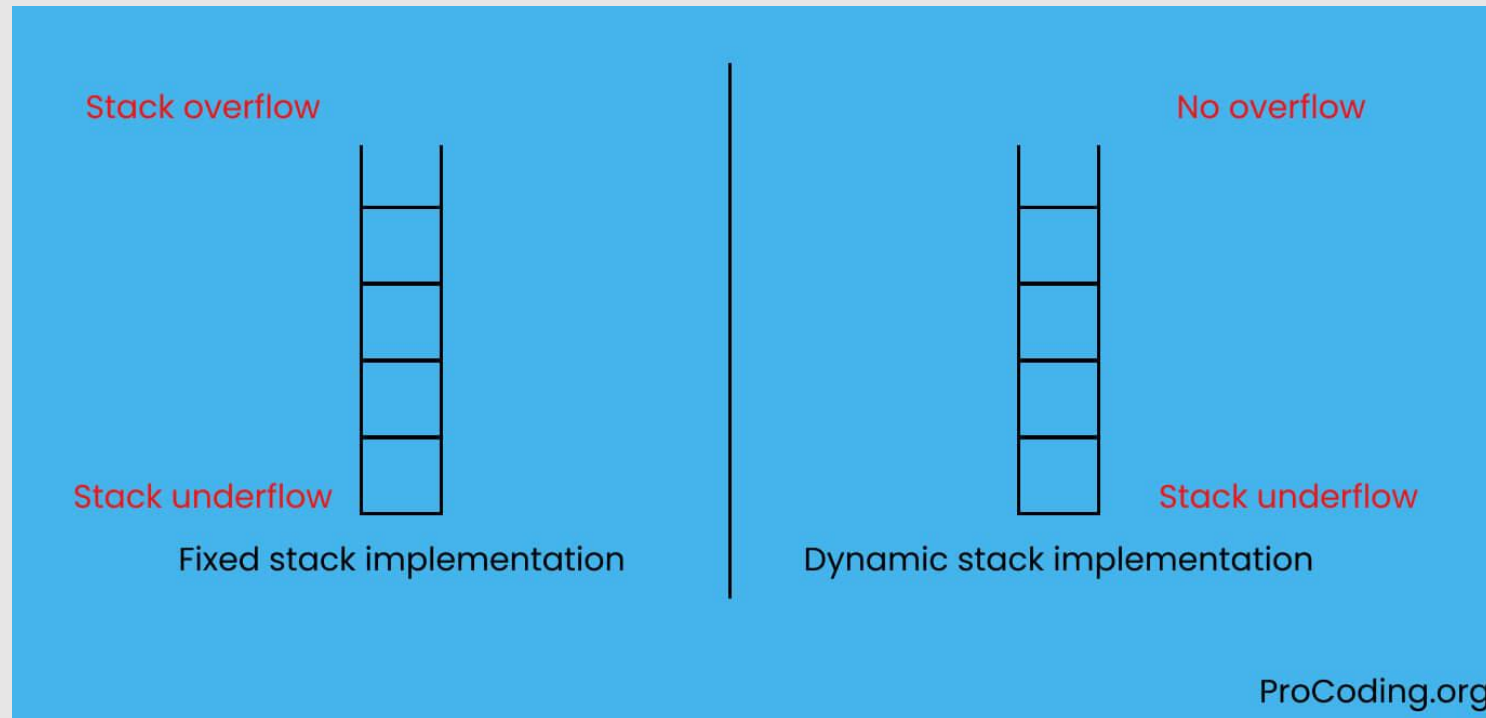
Stack underflow

Dynamic stack implementation



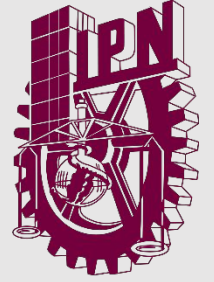
Considerations

Stack overflow happens when we try to push one more item onto our stack than it can actually hold.

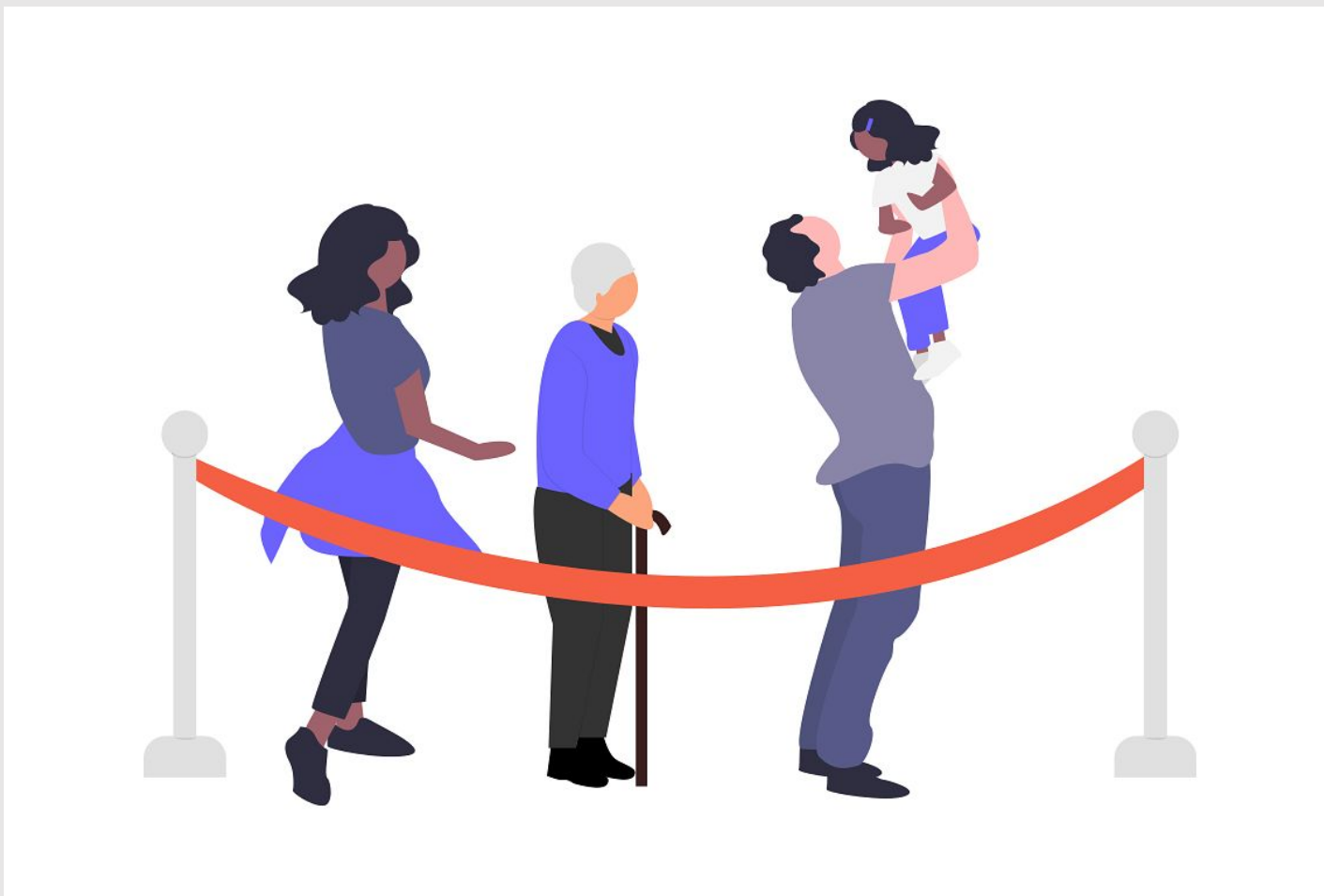


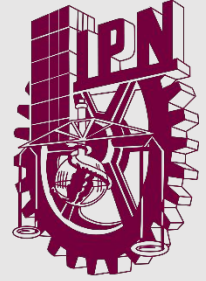
Stack underflow happens when we try to pop an item from the stack, when nothing is actually there to remove.

HW



- Investigar como evaluar expresiones aritmeticas con pilas

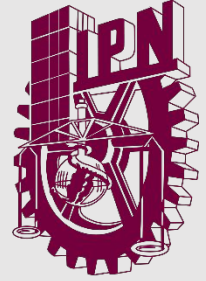




Queues

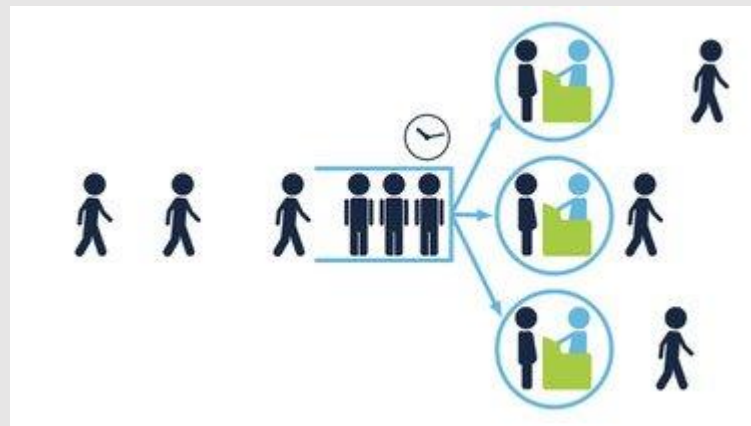
- **Definition:** A queue is a homogeneous collection of elements in which deletions can take place only at the front end, known as dequeue and insertions can take place only at the rear end, known as enqueue.

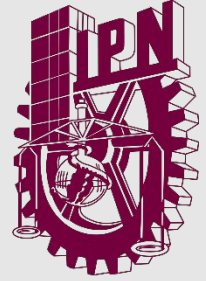




Queues

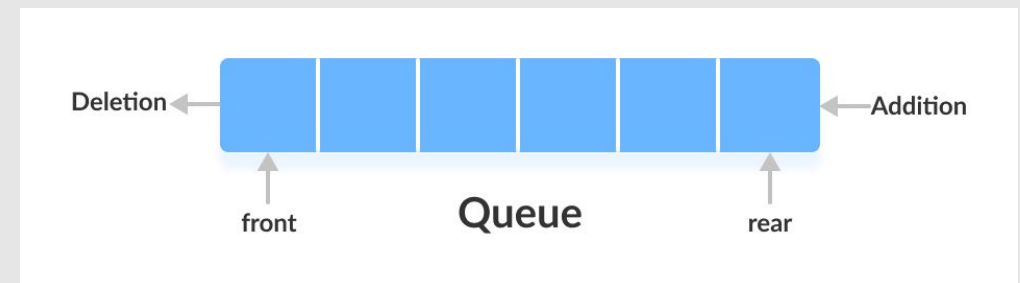
- A queue is a linear list in which data can only be inserted at one end, called the rear, and deleted from the other end, called the front. These restrictions ensure that the data are processed through the queue in the order in which they are received.
- The element to enter the queue first will be deleted from the queue first. That is why a queue is called First-In-First-Out (FIFO) system.

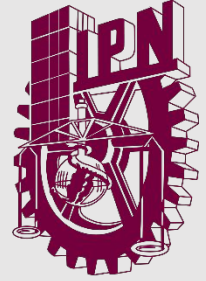




Queue Operations

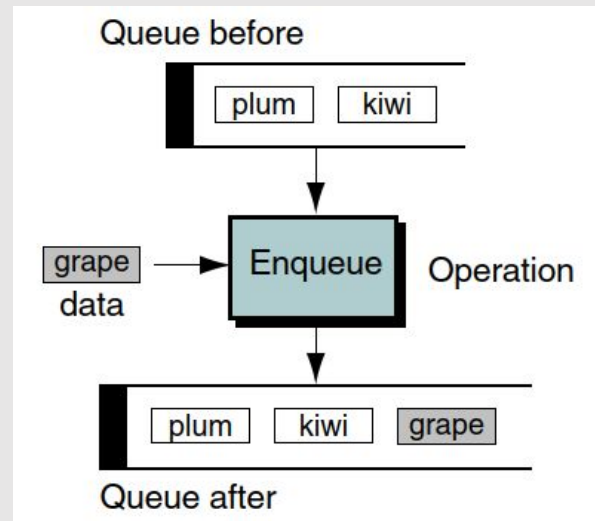
- There are four basic queue operations. Data can be inserted at the rear, deleted from the front, retrieved from the front, and retrieved from the rear.

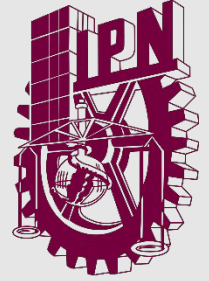




Enqueue

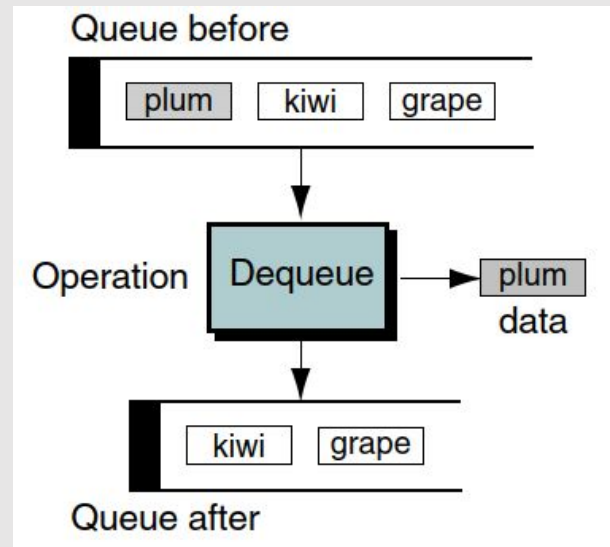
- The queue insert operation is known as enqueue. After the data have been inserted into the queue, the new element becomes the rear. As we saw with stacks, the only potential problem with enqueue is running out of room for the data. If there is not enough room for another element in the queue, the queue is in an overflow state.

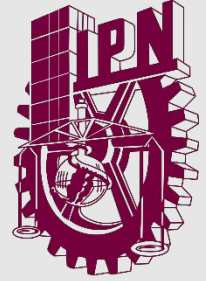




Deque

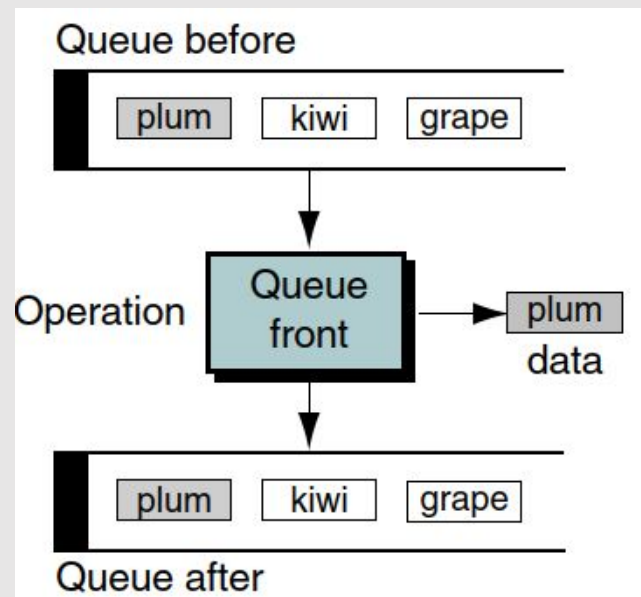
- The queue delete operation is known as dequeue. The data at the front of the queue are returned to the user and removed from the queue. If there are no data in the queue when a dequeue is attempted, the queue is in an underflow state.

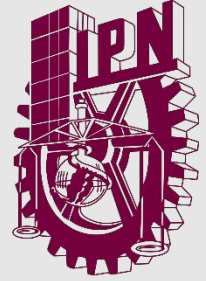




Queue Front

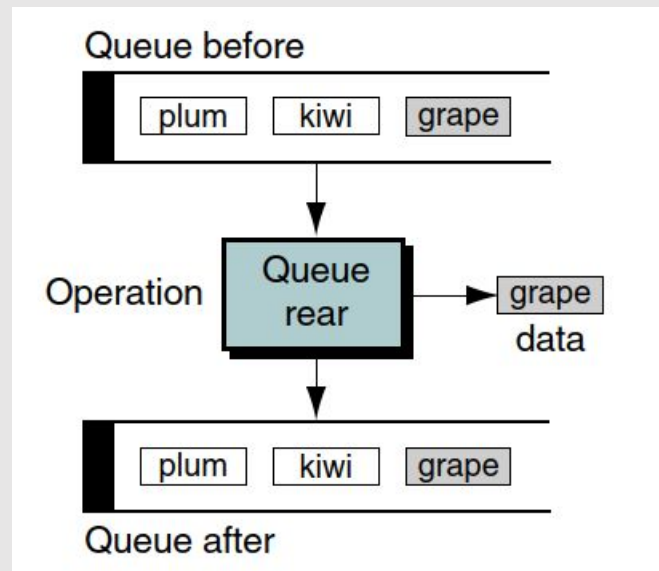
- Data at the front of the queue can be retrieved with queue front. It returns the data at the front of the queue without changing the contents of the queue. If there are no data in the queue when a queue front is attempted, then the queue is in an underflow state.

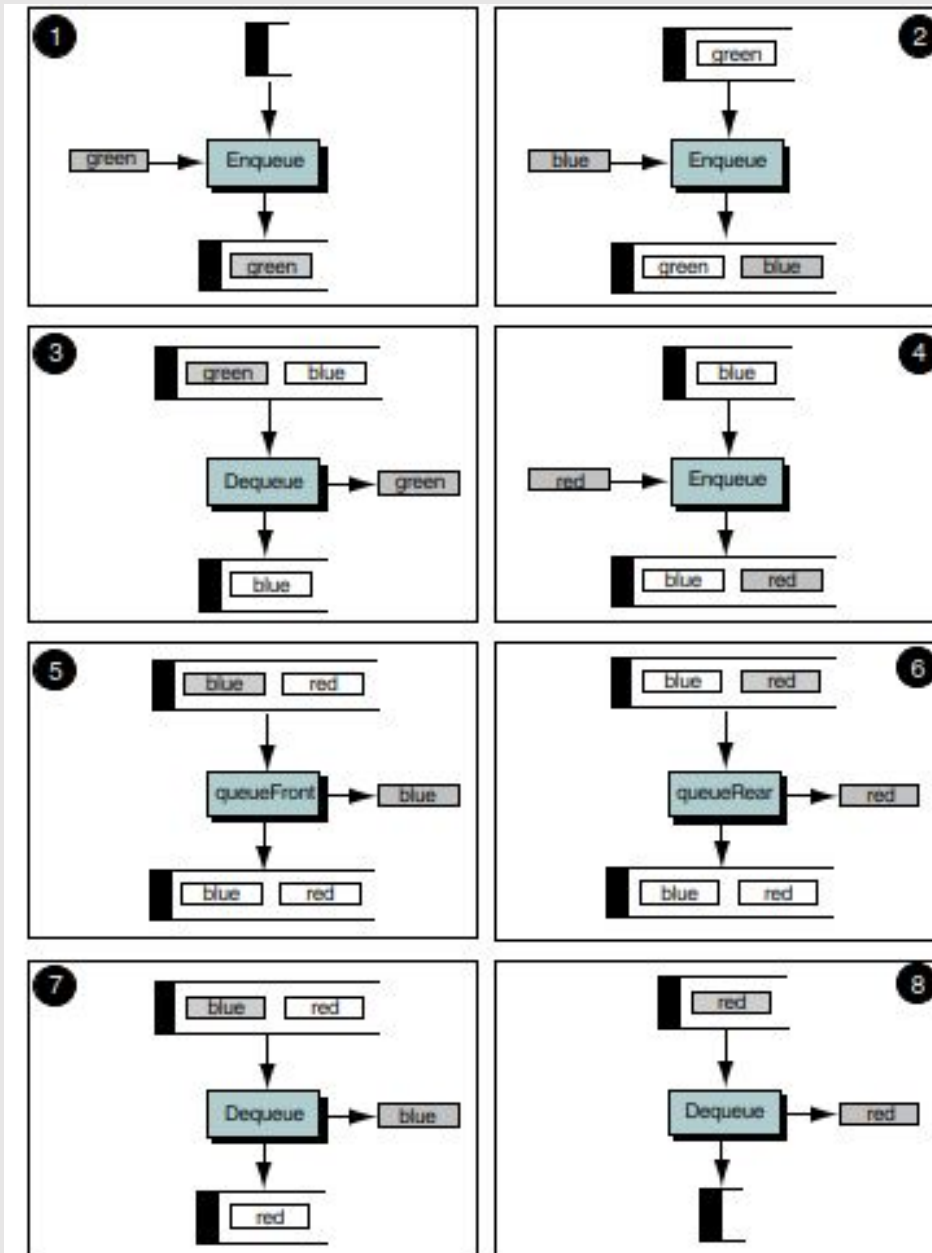
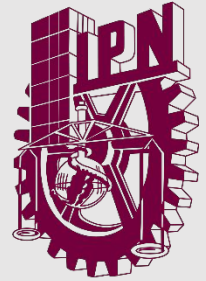




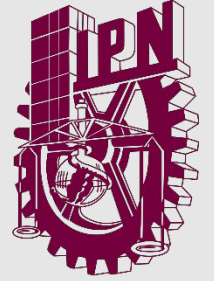
Queue Rear

- A parallel operation to queue front retrieves the data at the rear of the queue. It is known as queue rear. As with queue front, if there are no data in the queue when a queue rear is attempted, the queue is in an underflow state.

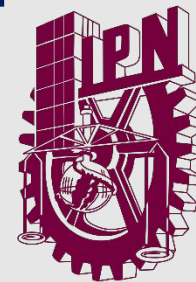


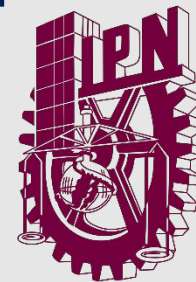


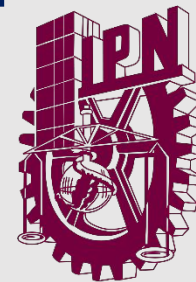
Tarea

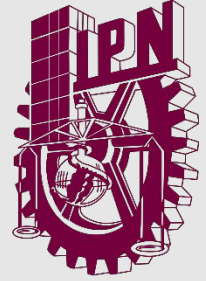


- De la estructura de datos cola:
 - Investigar otras operaciones
 - Implementación dinámica
 - Implementación estática



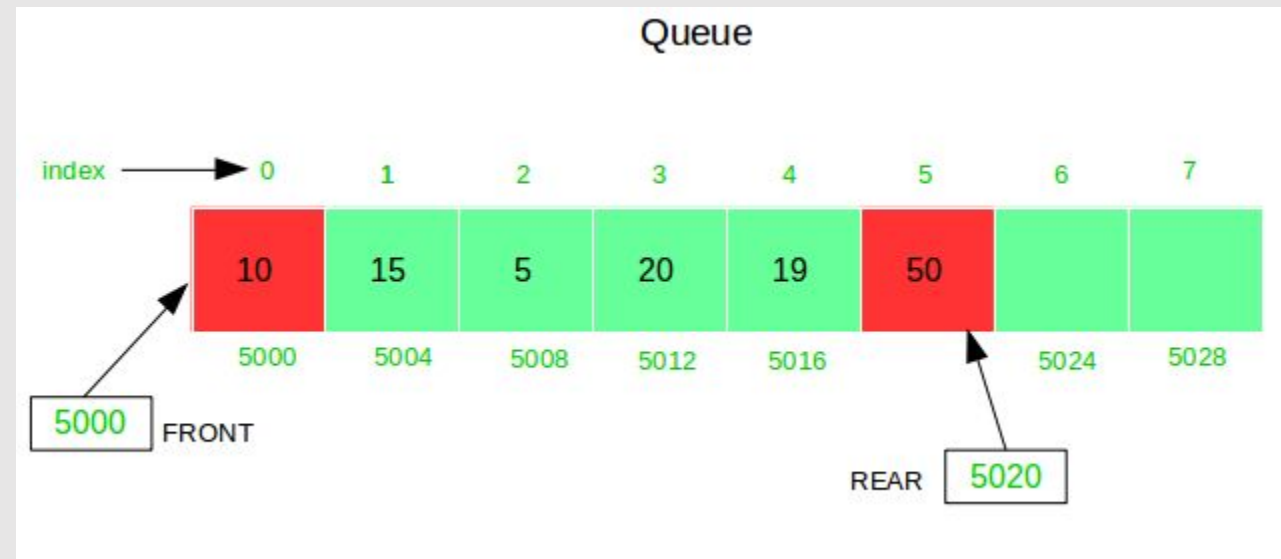


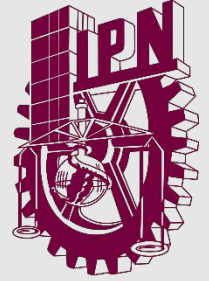




REPRESENTATION OF QUEUE

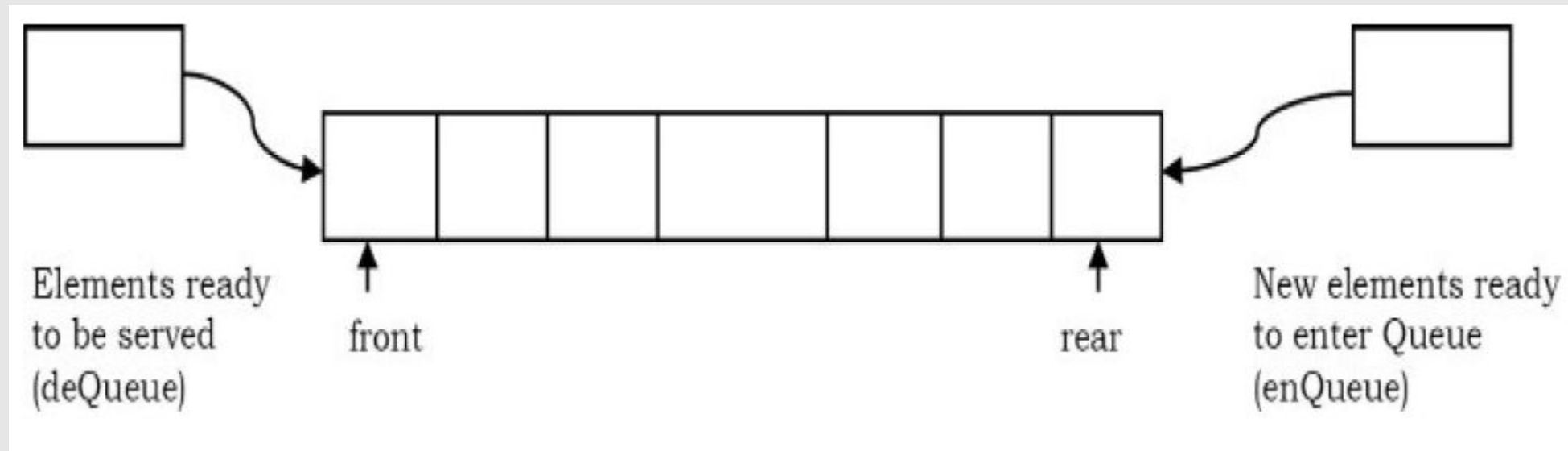
Static implementation : Static implementation means array implementation. In this implementation, we should be assured of the number of elements we want in the queue. The size of the array is decided before execution at compile time. Once the array is declared, its size cannot be altered. The array at run time neither enlarged nor shrunk. The beginning of the array, that is, 0th element is the front and the last memory location will be rear of the queue. A queue can be declared as an array of any data type.

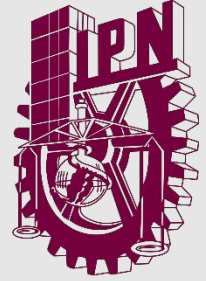




REPRESENTATION OF QUEUE

- Dynamic implementation is carried out using pointers. By applying increment operation on pointer successive locations of memory can be accessed and an element can be stored in that location.

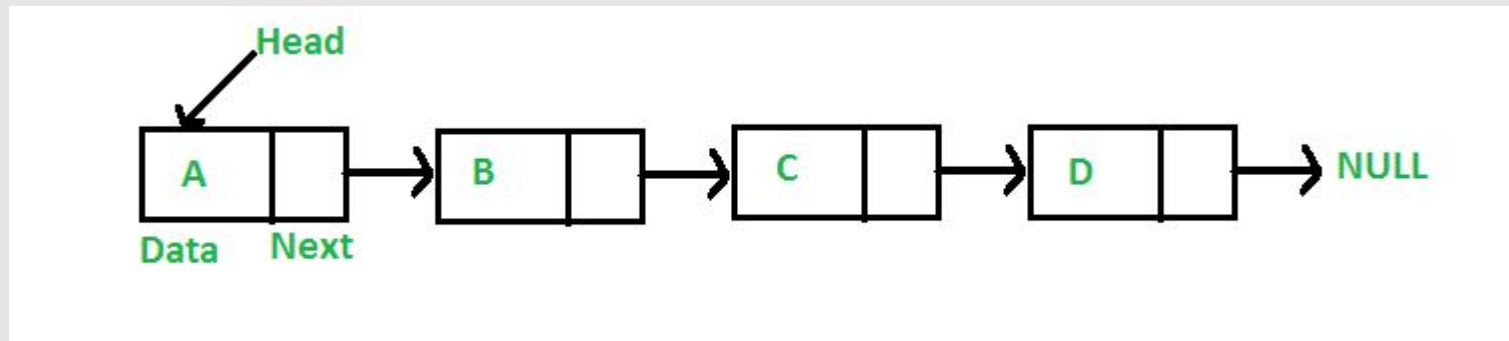


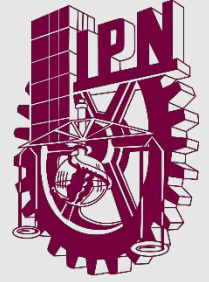


Introduction

The linked list is a linear data structure where each node has two parts.

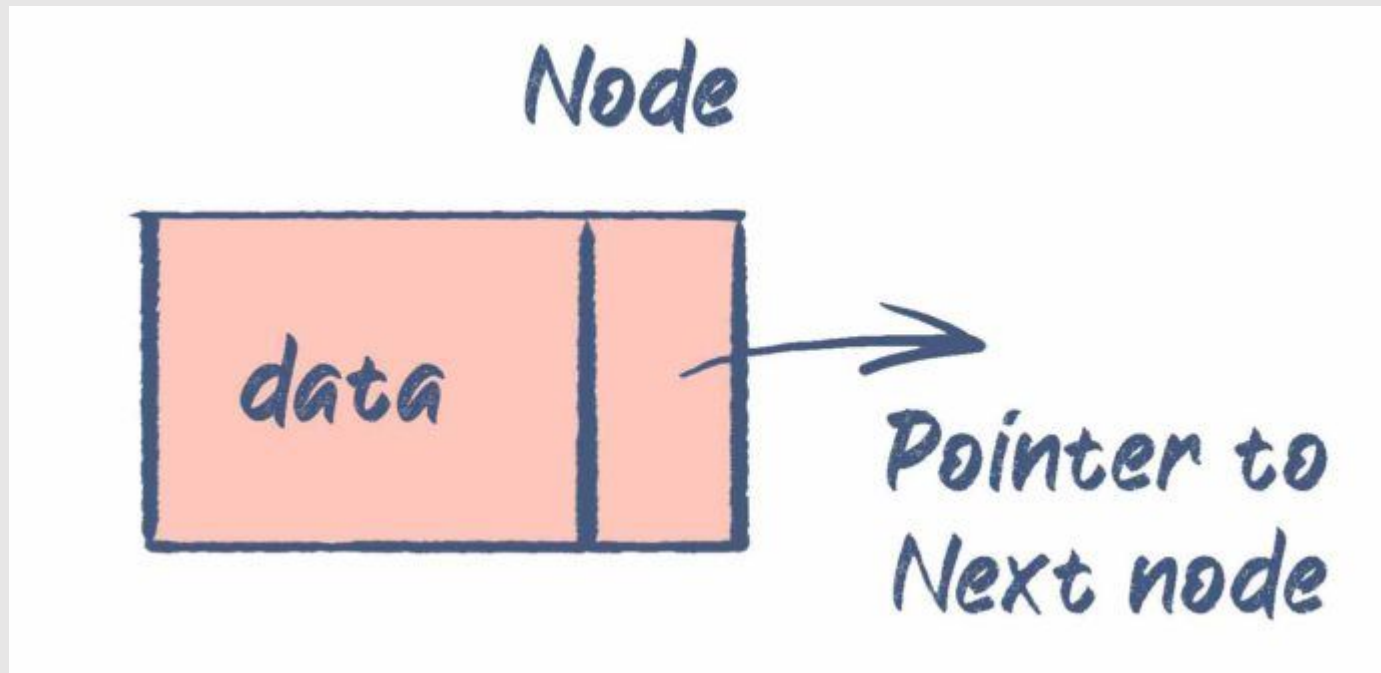
1. Data
2. Reference to the next node



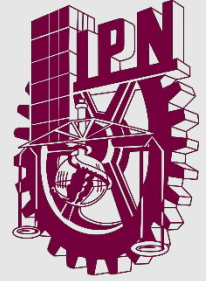


Node

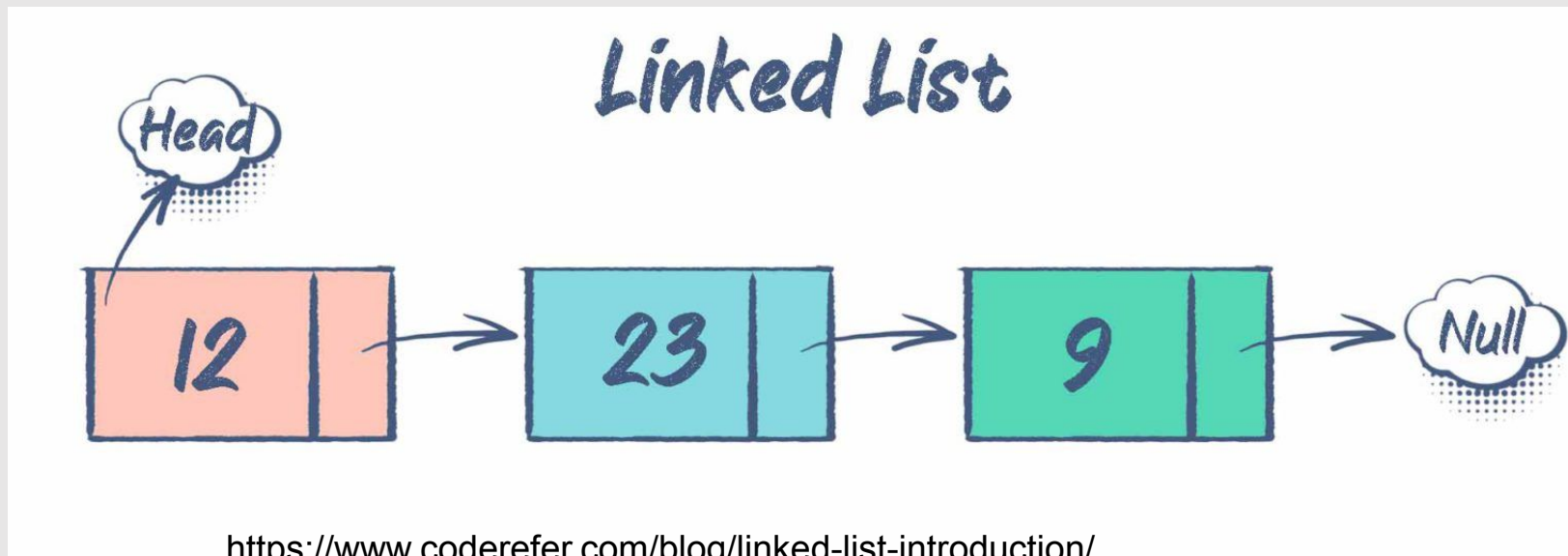
- A node is a part of linked list which consists of two things – data and pointer / address to next node.



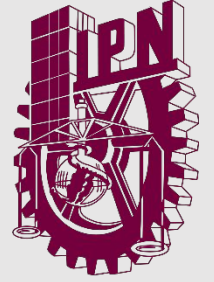
Linked List



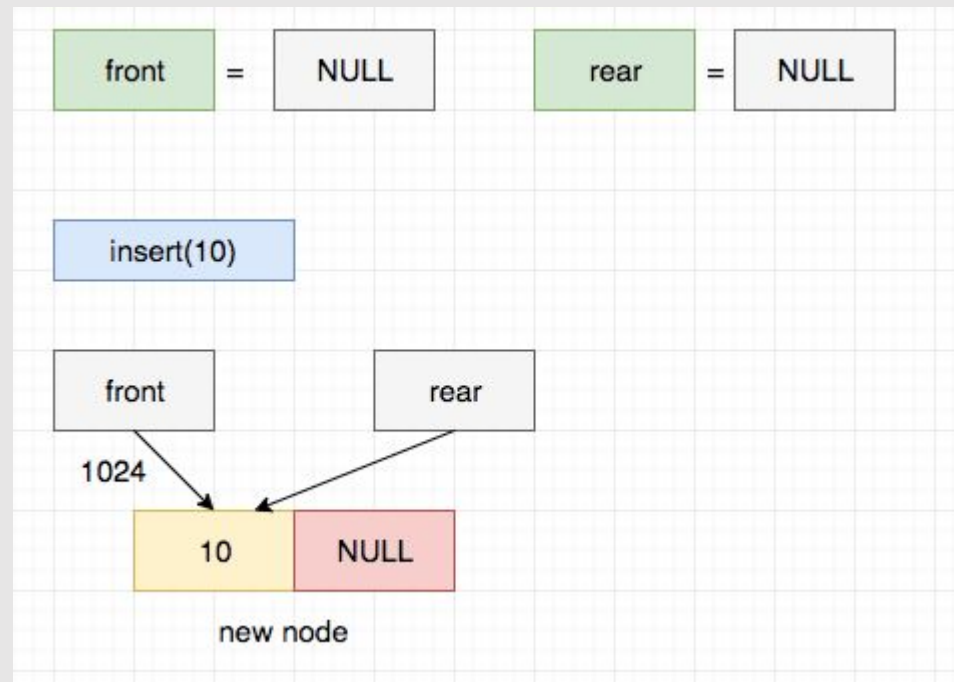
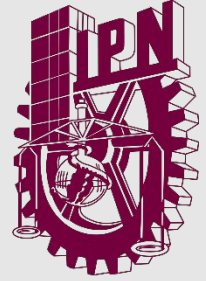
- A linked list is a Linear data structure which consists of chain of nodes each addressing to one another. The first element of LinkedList is usually considered as head. The last element of the linkedlist refers to null meaning that the linkedlist terminates at that node.

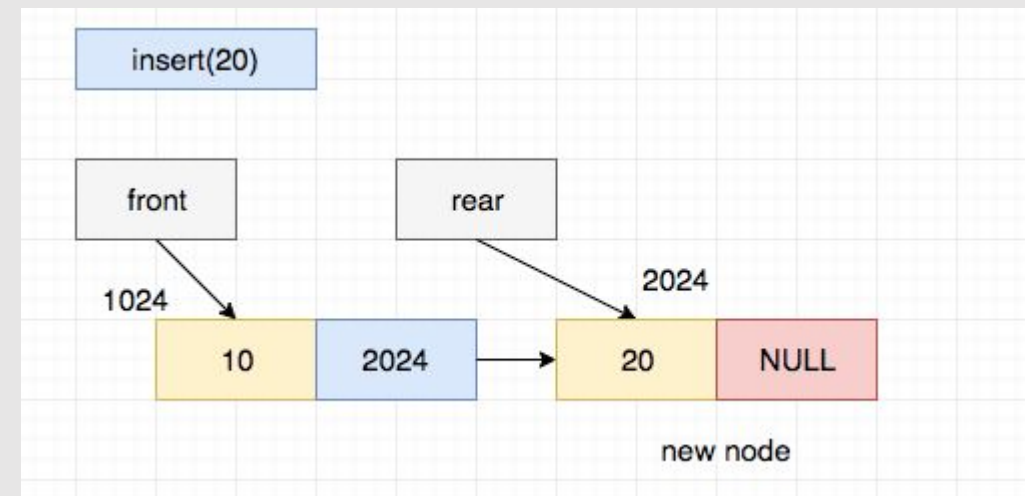
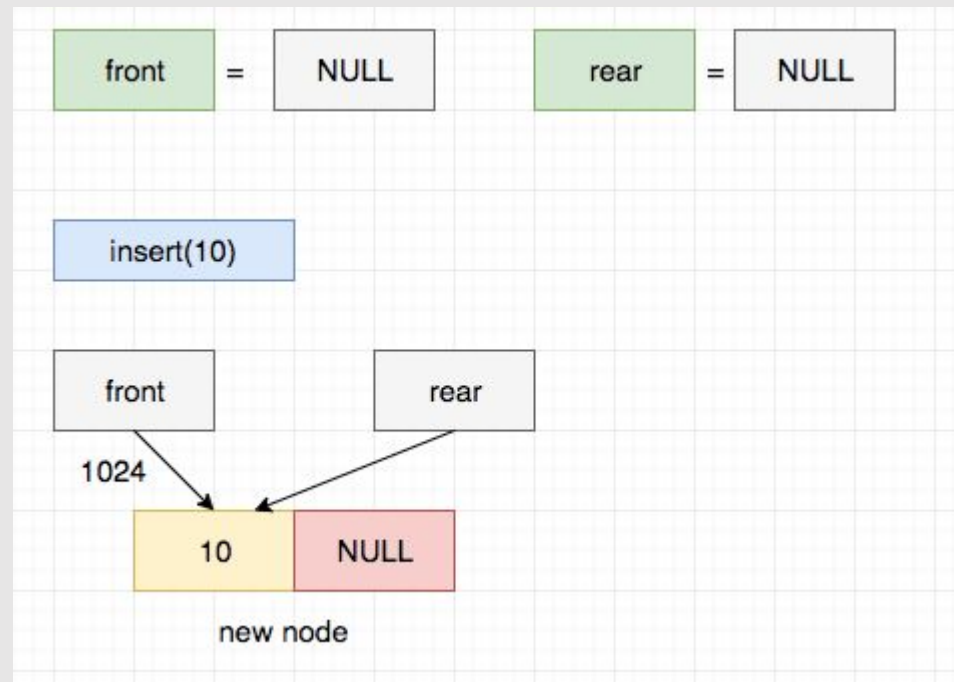
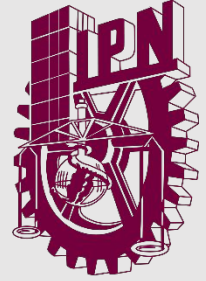


HW



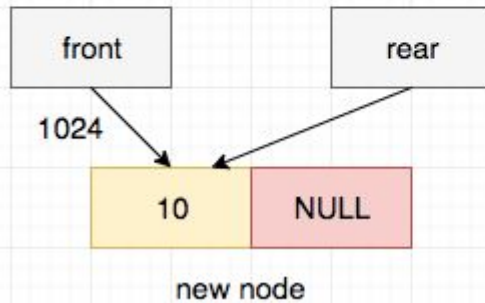
- Advantages and disadvantages



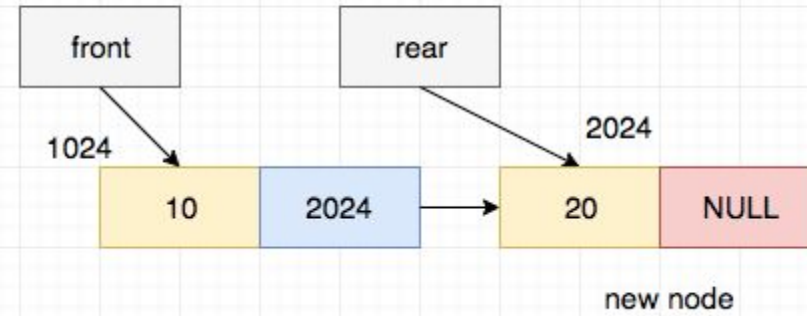


front = NULL rear = NULL

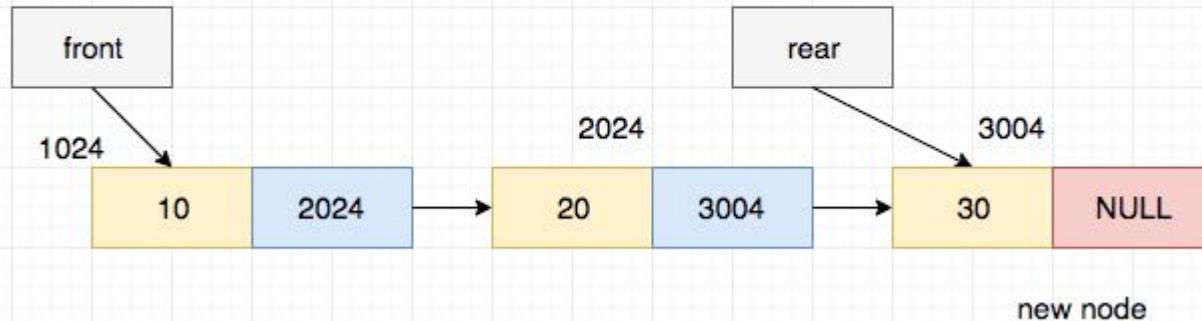
insert(10)

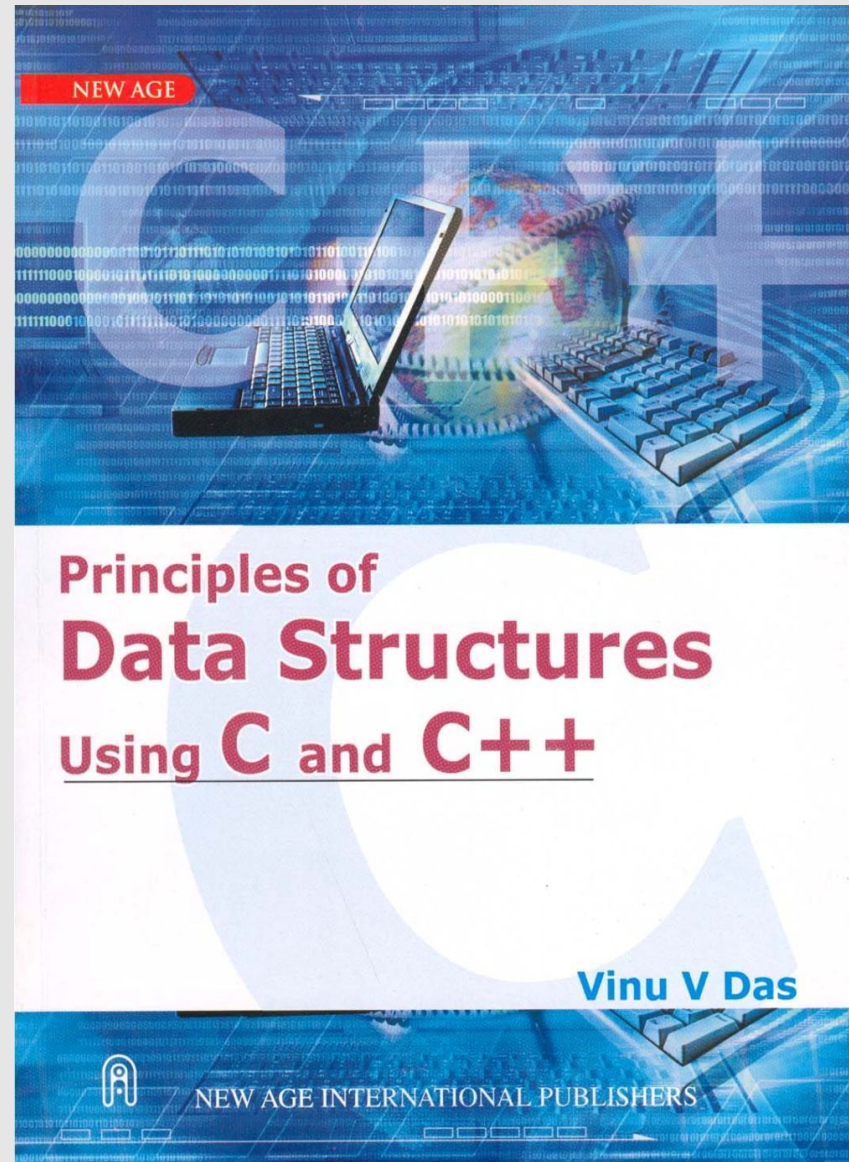
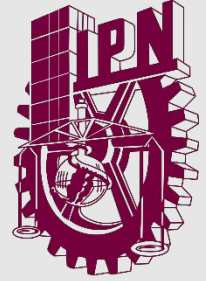


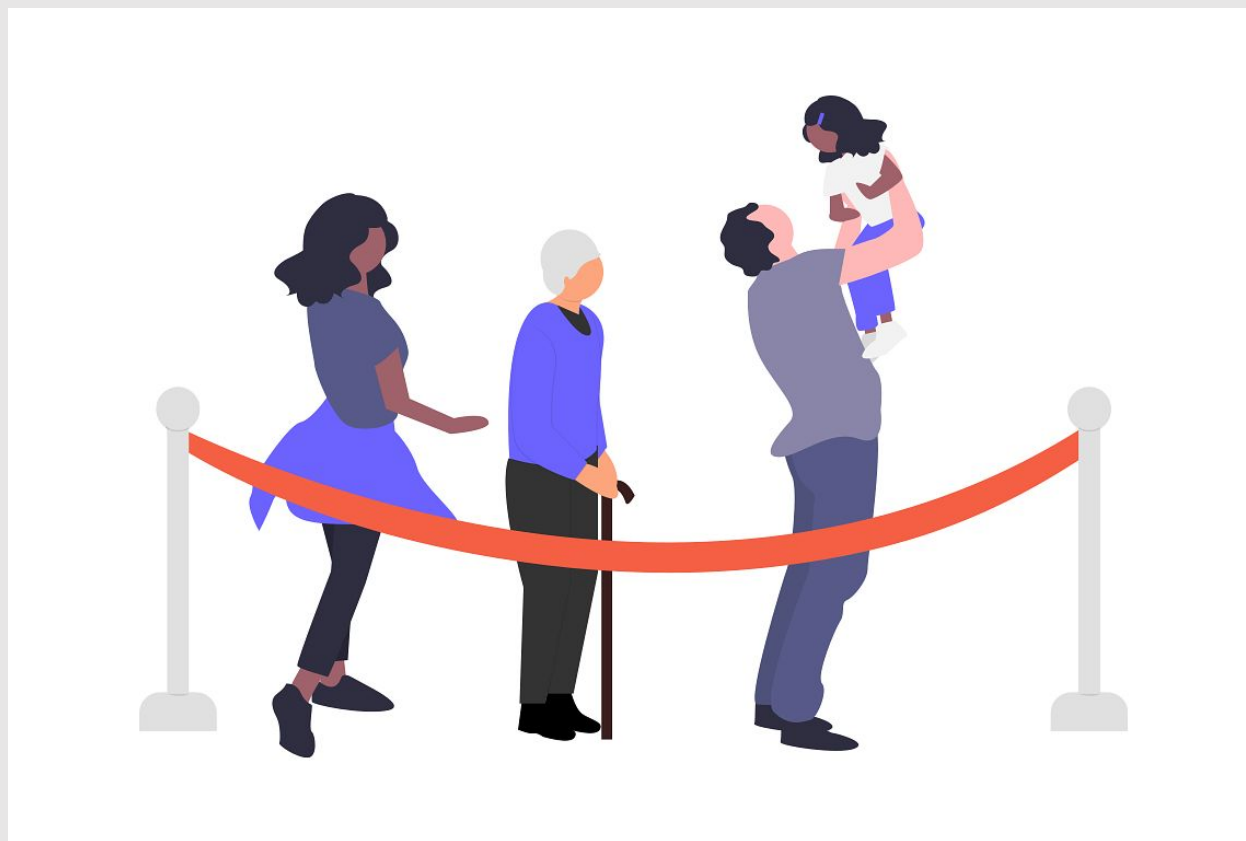
insert(20)

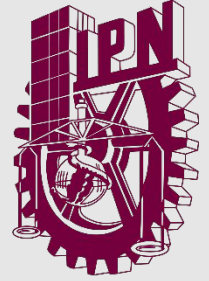


insert(30)





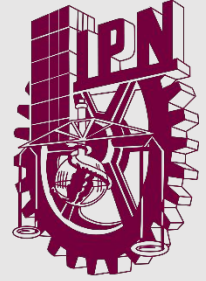




PRIORITY QUEUES

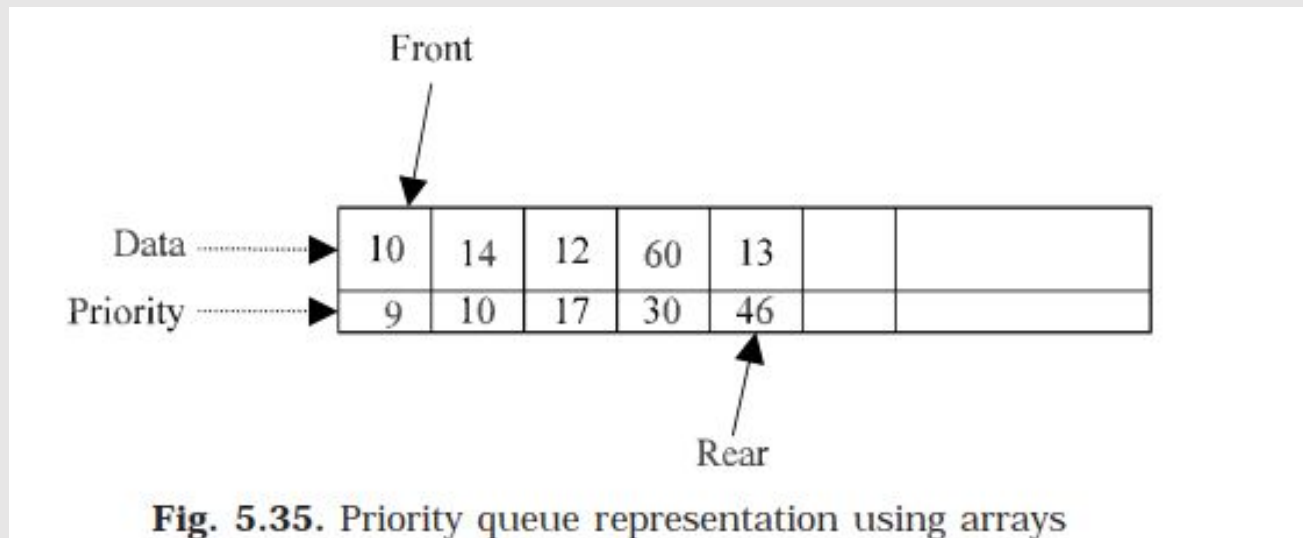
Priority Queue is a queue where each element is assigned a priority. In priority queue, the elements are deleted and processed by following rules.

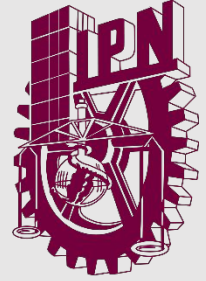
1. An element of higher priority is processed before any element of lower priority.
2. Two elements with the same priority are processed according to the order in which they were inserted to the queue.



PRIORITY QUEUES

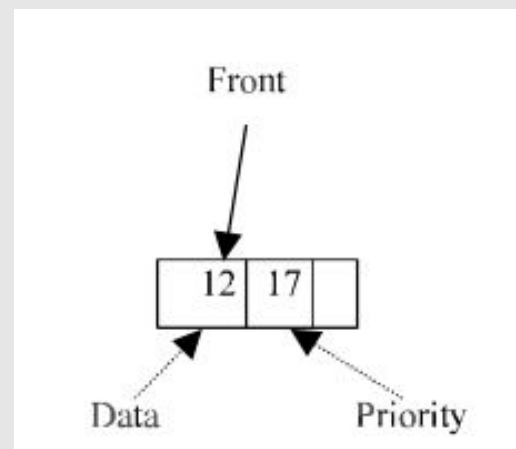
- Figure gives a pictorial representation of priority queue using arrays after adding 5 elements (10,14,12,60,13) with its corresponding priorities (9,10,17,30,46). Here the priorities of the data(s) are in ascending order.

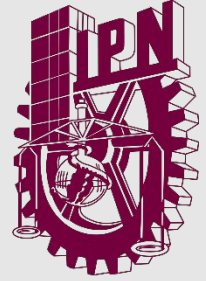




Implementation

- It is always better to implement the priority queue using linked list.
- A node in the priority queue will contain DATA, PRIORITY and NEXT field. DATA field will store the actual information; PRIORITY field will store its corresponding priority of the DATA and NEXT will store the address of the next node.





Implementation (2)

When an element is inserted into the priority queue, it will check the priority of the element with the element(s) present in the linked list to find the suitable position to insert.

The node will be inserted in such a way that the data in the priority field(s) is in ascending order.

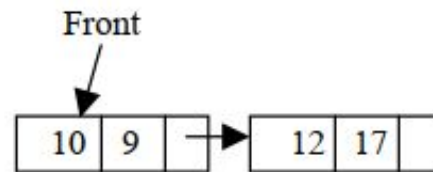
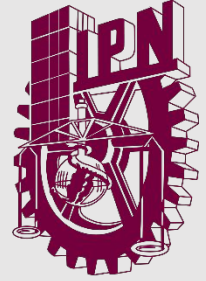


Fig. 5.37. push(DATA =10, PRIORITY = 9)



Implementation (2)

When an element is inserted into the priority queue, it will check the priority of the element with the element(s) present in the linked list to find the suitable position to insert.

The node will be inserted in such a way that the data in the priority field(s) is in ascending order.

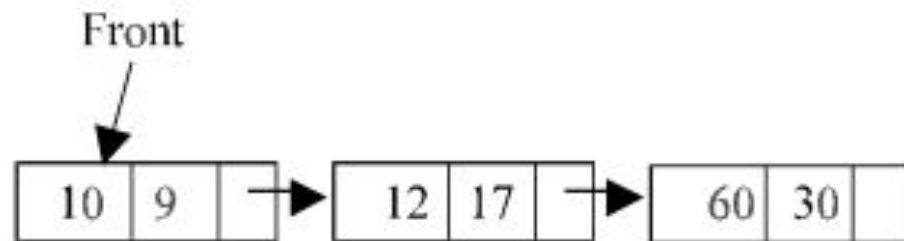


Fig. 5.38. push(DATA = 60, PRIORITY = 30)

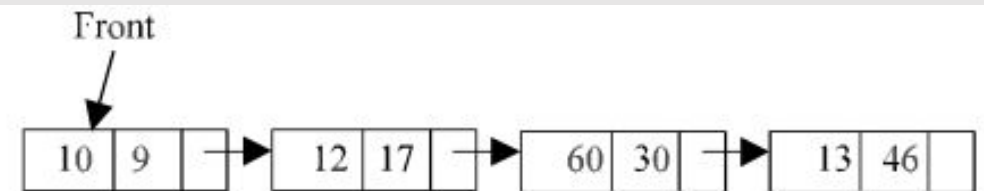
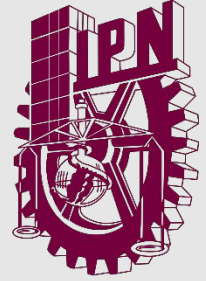


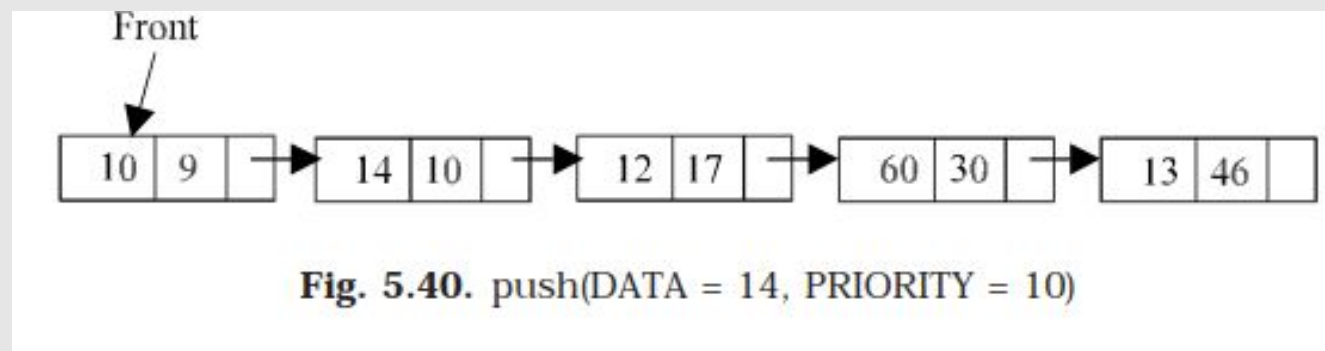
Fig. 5.39. push(DATA = 13, PRIORITY = 46)

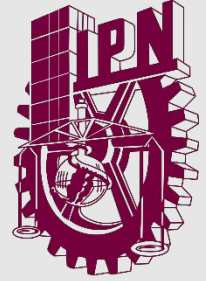


Implementation (2)

When an element is inserted into the priority queue, it will check the priority of the element with the element(s) present in the linked list to find the suitable position to insert.

The node will be inserted in such a way that the data in the priority field(s) is in ascending order.





Implementation (2)

When an element is inserted into the priority queue, it will check the priority of the element with the element(s) present in the linked list to find the suitable position to insert.

The node will be inserted in such a way that the data in the priority field(s) is in ascending order.

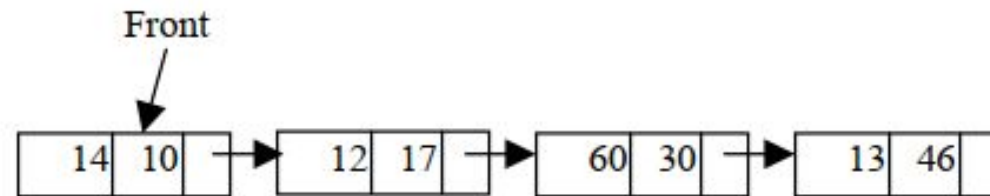


Fig. 5.41. $x = \text{pop}()$ (i.e., 10)

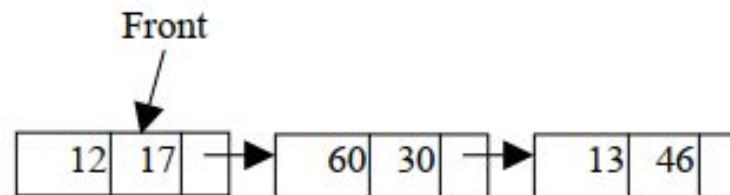
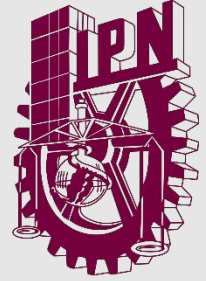


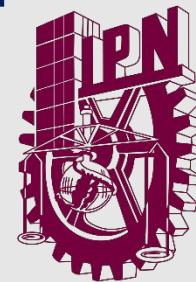
Fig. 5.42. $x = \text{pop}()$ (i.e., 14)

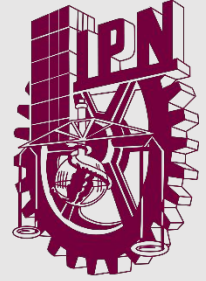


Tabla

- Realizar una tabla como la que se muestra, poner las operaciones que han revisado y buscar su complejidad para diferentes implementaciones.
- Escribir una conclusión a partir de lo observado.

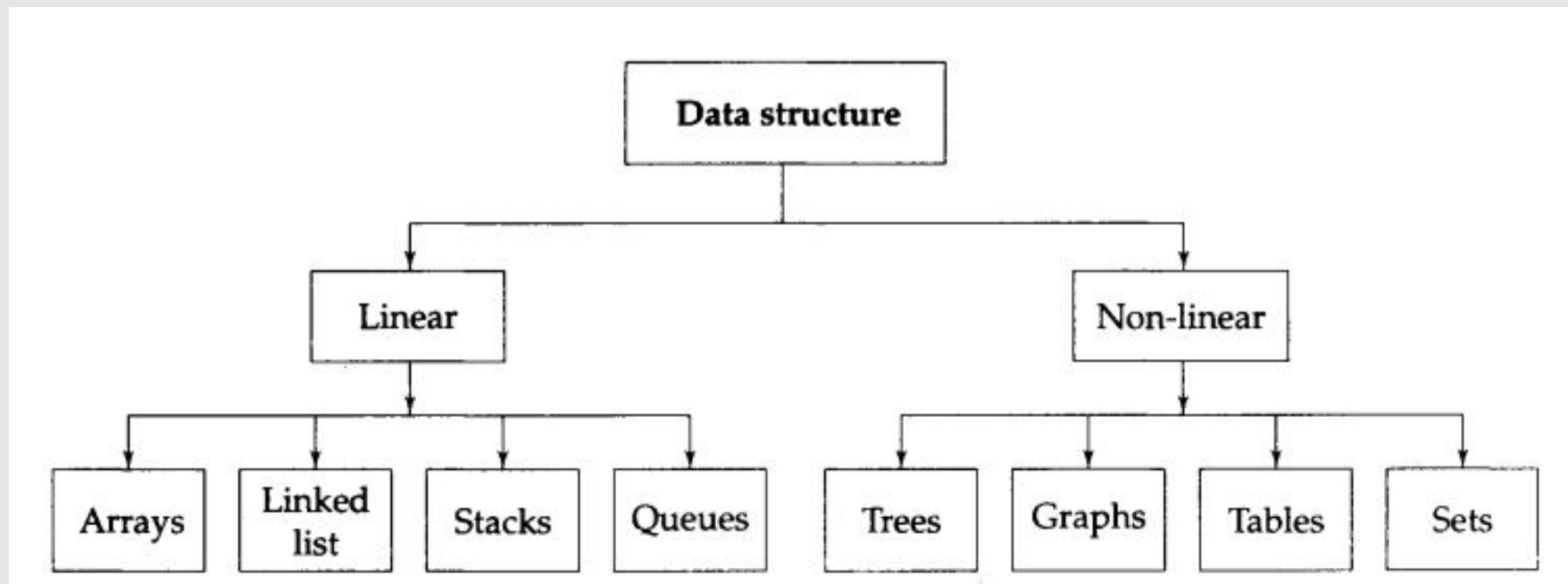
Implementación	Operación	Complejidad

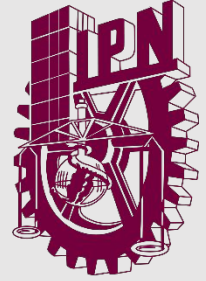




Data structures

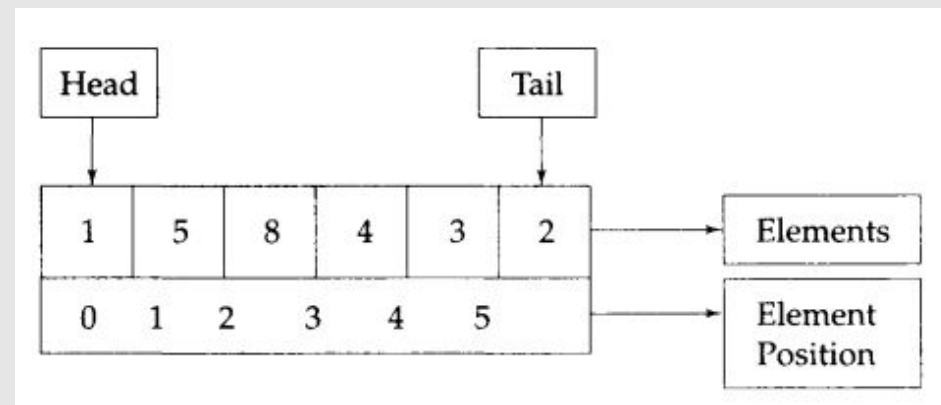
Data structures have been classified in several ways.

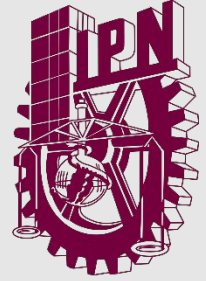




List

- A list is a series of linearly arranged numbers of same type. The list can be of basic data type or custom data type. The elements are positioned one after another and their position number appears in sequence. The first element of the list is called HEAD and the last element is called as TAIL.

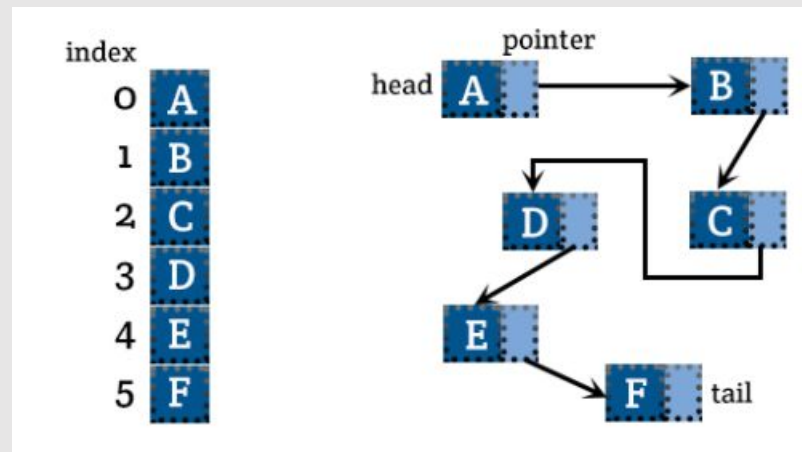


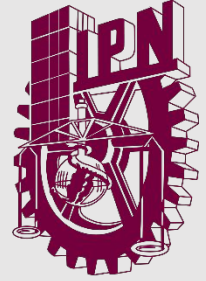


Implementation of a list

There are two methods to implement the list: static and dynamic.

1. Static implementation: Static implementation can be achieved using arrays.
2. Dynamic implementation: Pointer can also be used for this implementation.

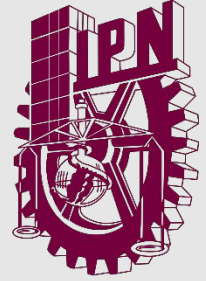




Basic operations

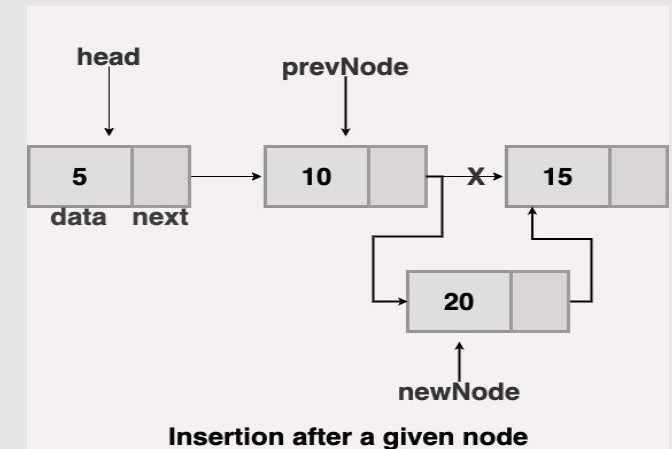
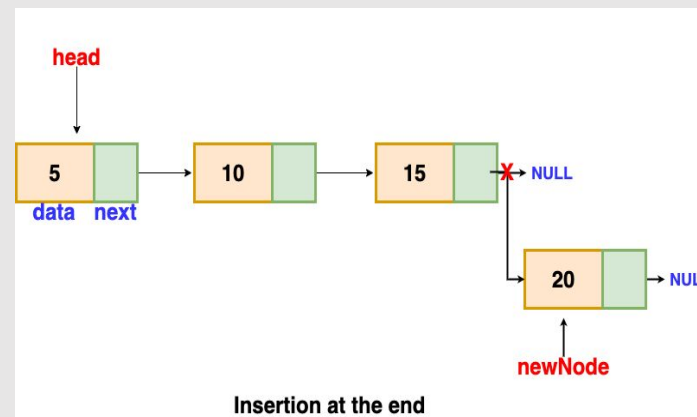
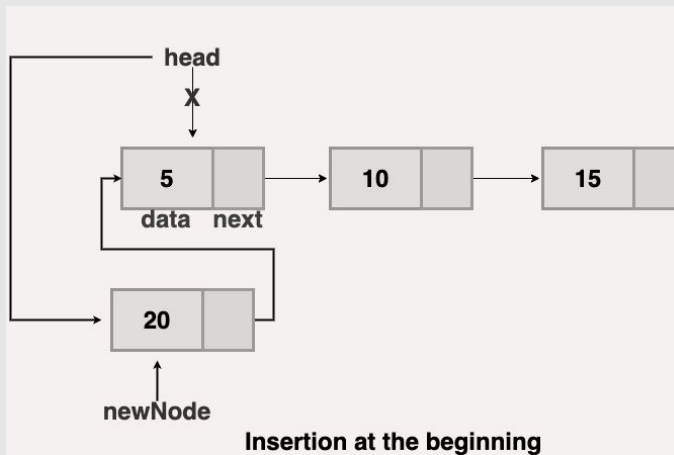
- The four basic list operations are insertion, deletion, retrieval, and traversal. Insertion is used to add a new element to the list. Deletion is used to remove an element from the list. Retrieval is used to get the information related to an element without changing the structure of the list. Traversal is used to traverse the list while applying a process to each element.

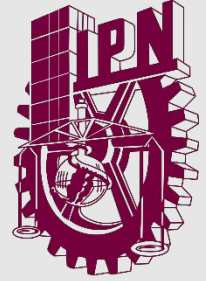




Insertion

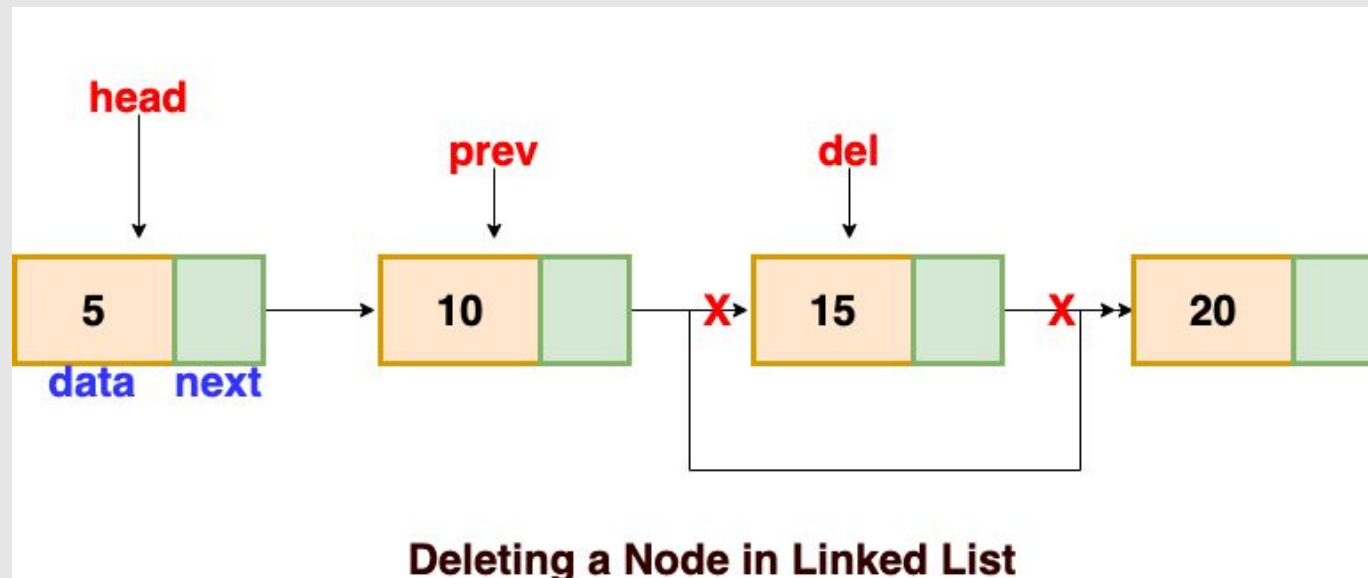
- List insertion can be ordered or random. Ordered lists are maintained in sequence according to the data or, when available, a key that identifies the data. A key is one or more fields within a structure that identifies the data.



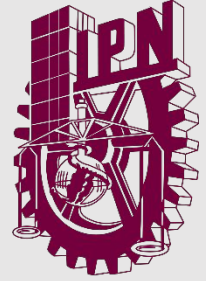


Deletion

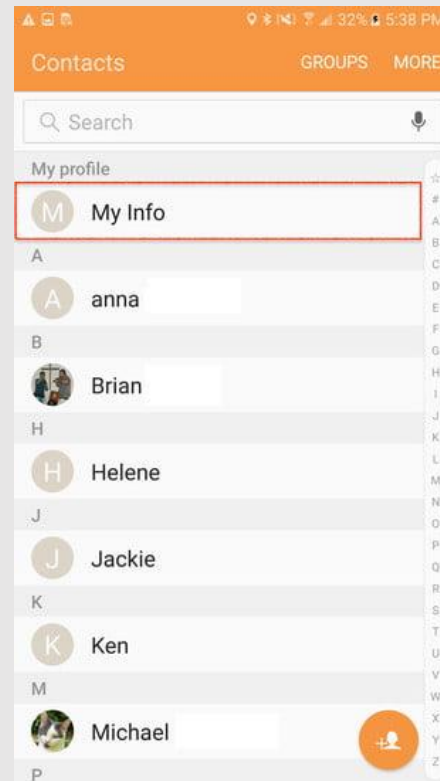
- Deletion from a list requires that the list be searched to locate the data being deleted. Once located, the data are removed from the list.

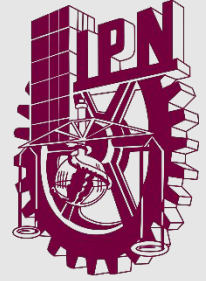


Retrieval



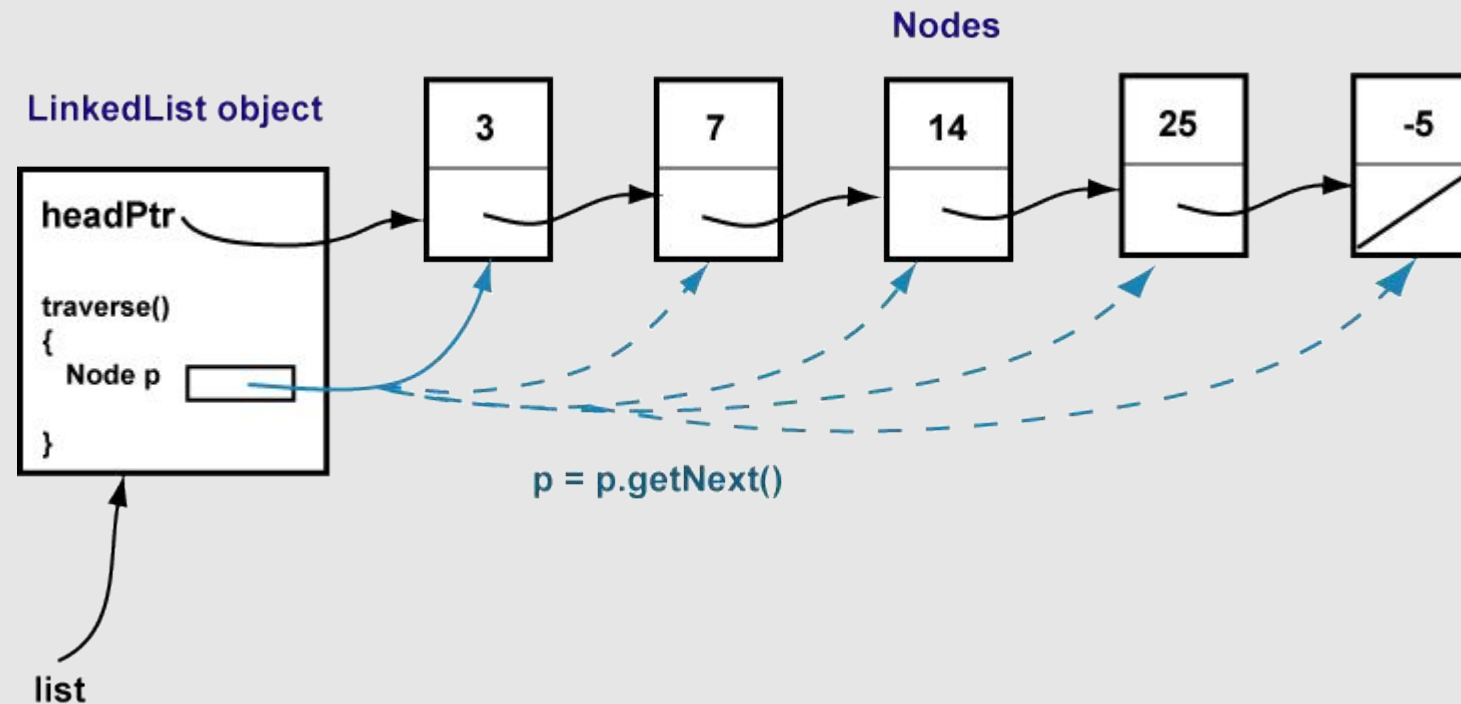
- List retrieval requires that data be located in a list and presented to the calling module without changing the contents of the list.

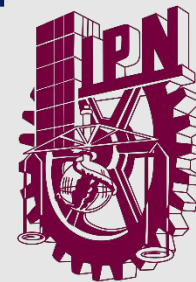




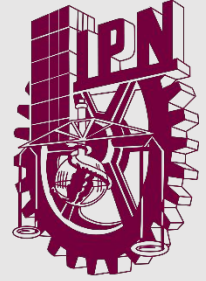
Traversal

- List traversal processes each element in a list in sequence. It requires a looping algorithm rather than a search.

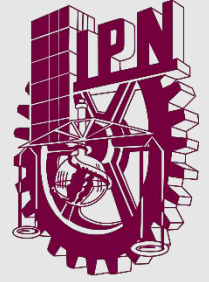




HW

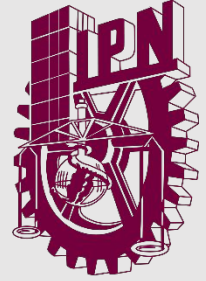


- De la implementación descrita, implementar 3 funciones más:
 1. Buscar elemento en la lista y que regrese el índice
 2. Recuperar un elemento de la lista a partir de un índice
 3. Conteo de elementos en la lista
 4. Destruir lista

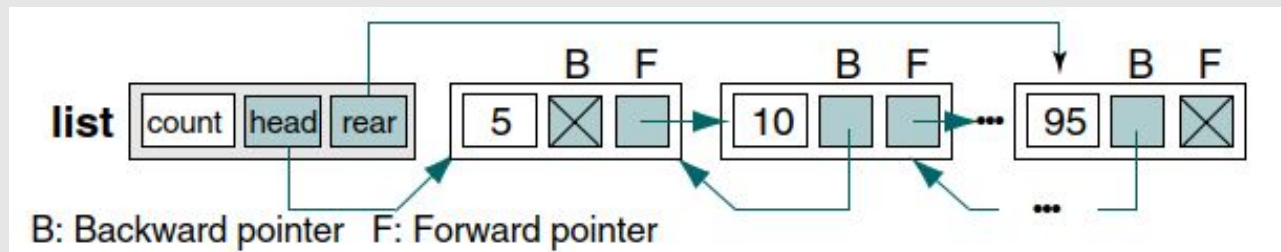


DOUBLY LINKED LISTS BE LIKE



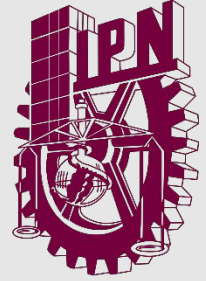


- One of the most powerful implementations is the doubly linked list. A doubly linked list is a linked list structure in which each node has a pointer to both its successor and its predecessor.

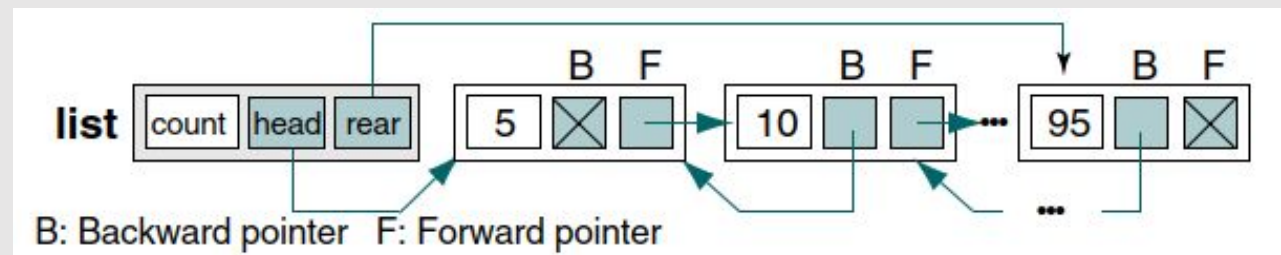
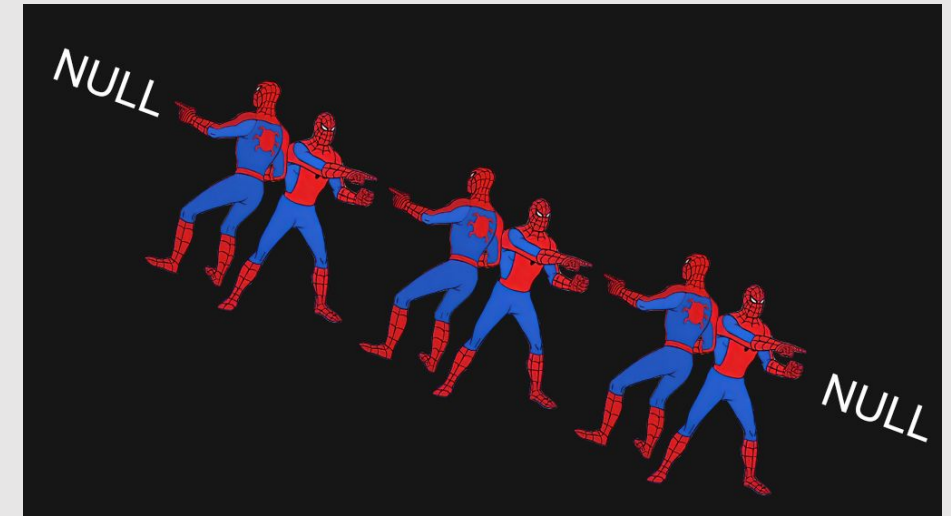


DOUBLY LINKED LISTS BE LIKE

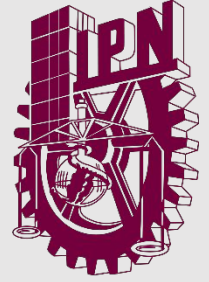




- One of the most powerful implementations is the doubly linked list. A doubly linked list is a linked list structure in which each node has a pointer to both its successor and its predecessor.



Each node contains two pointers: a backward pointer to its predecessor and a forward pointer to its successor.

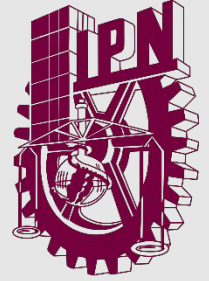


Main advantages and disadvantages

- The *advantage* of a doubly linked list is that given a node in the list, we can navigate in both directions.

The primary *disadvantages* of doubly linked lists are:

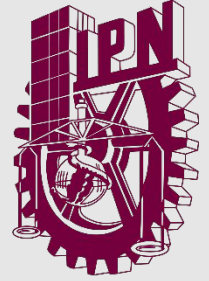
- Each node requires an extra pointer, requiring more space.
- The insertion or deletion of a node takes a bit longer (more pointer operations).



Struct

- Similar to a singly linked list, let us implement the operations of a doubly linked list.
- Following is a type declaration for a doubly linked list of integers:

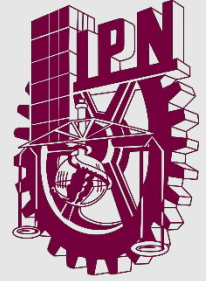
```
struct DLLNode {  
    int data;  
    struct DLLNode *next;  
    struct DLLNode *prev;  
};
```



Doubly Linked List Insertion

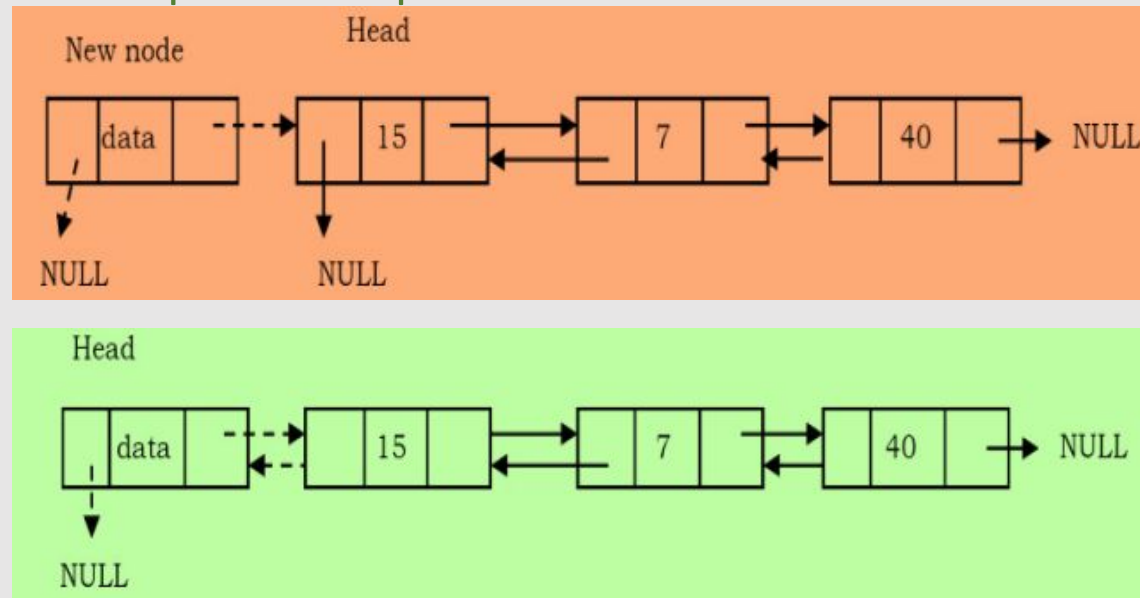
- Insertion into a doubly-linked list has three cases (same as singly linked list):
- Inserting a new node before the head.
- Inserting a new node after the tail (at the end of the list).
- Inserting a new node at the middle of the list.

Inserting a Node in Doubly Linked List at the Beginning

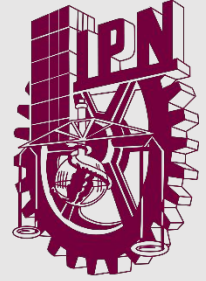


In this case, new node is inserted before the head node. Previous and next pointers need to be modified and it can be done in two steps:

- Update the right pointer of the new node to point to the current head node (dotted link in below figure) and also make left pointer of new node as NULL.
- Update head node's left pointer to point to the new node and make new node as head.

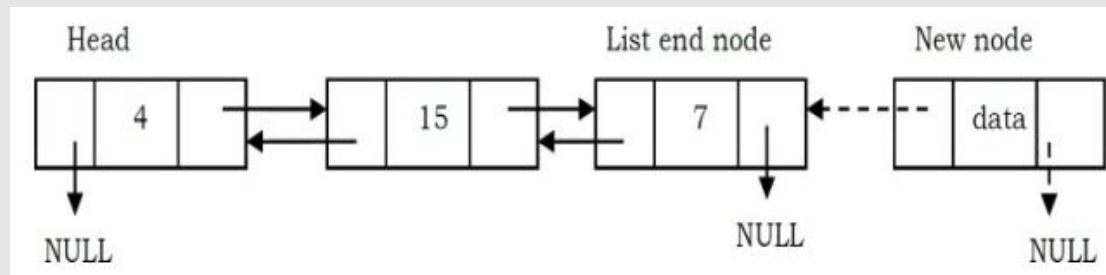


Inserting a Node in Doubly Linked List at the Ending

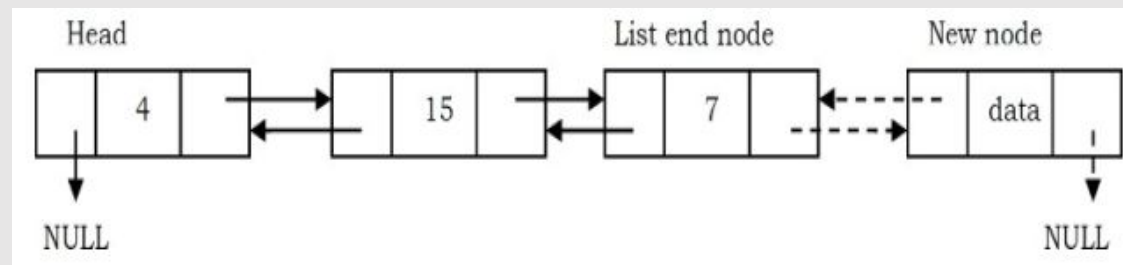


In this case, traverse the list till the end and insert the new node.

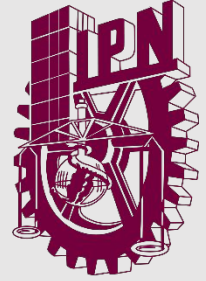
- New node right pointer points to NULL and left pointer points to the end of the list.



- Update right pointer of last node to point to new node.

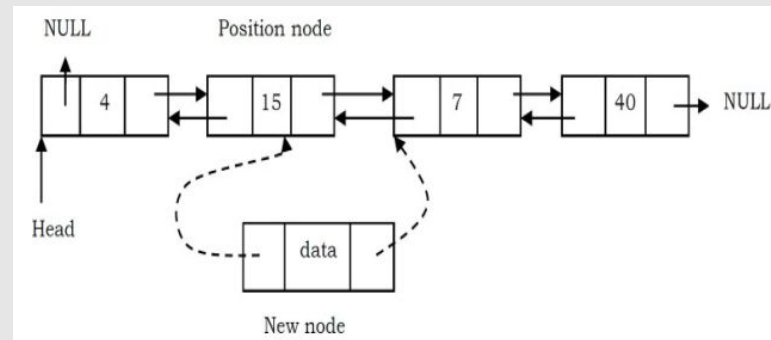


Inserting a Node in Doubly Linked List at the Middle

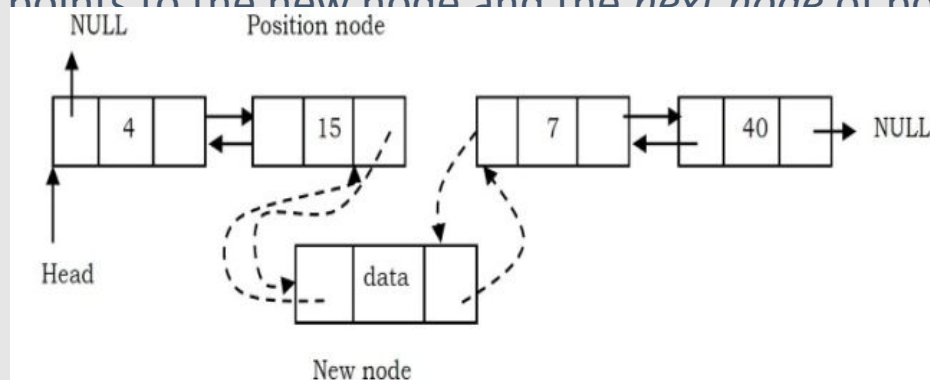


Traverse the list to the position node and insert the new node

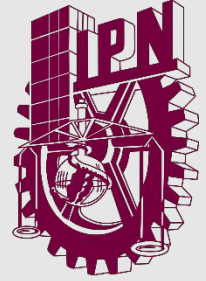
- *New node* right pointer points to the next node of the *position node* where we want to insert the new node. Also, *new node* left pointer points to the *position node*.



- Position node right pointer points to the new node and the next node of position node left pointer points to new node.

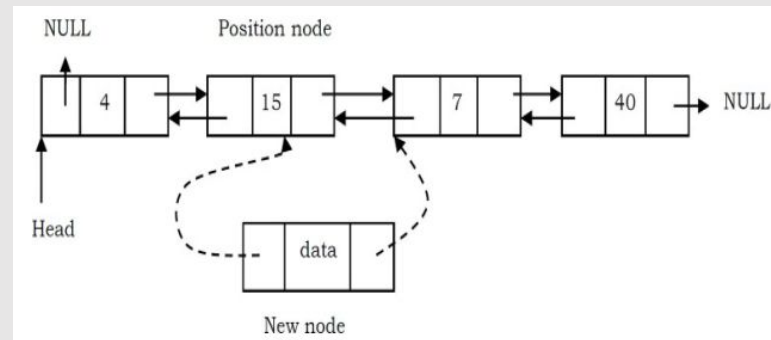


Inserting a Node in Doubly Linked List at the Middle

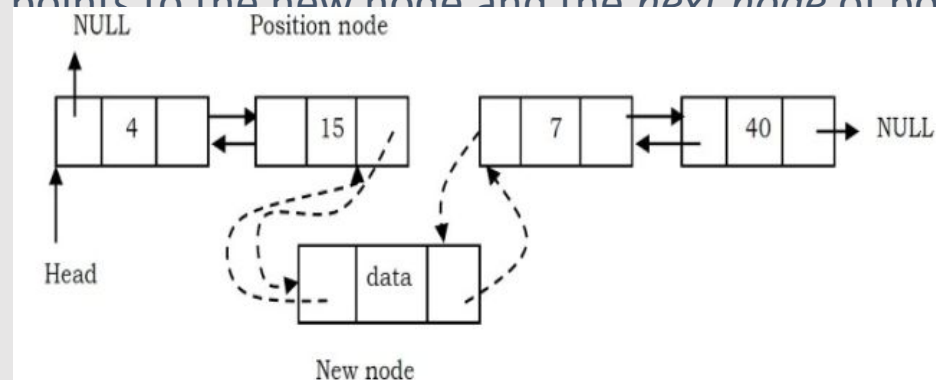


Traverse the list to the position node and insert the new node

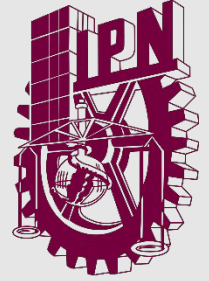
- *New node* right pointer points to the next node of the *position node* where we want to insert the new node. Also, *new node* left pointer points to the *position node*.



- Position node right pointer points to the new node and the next node of position node left pointer points to new node.



Time Complexity: $O(n)$. In the worst case

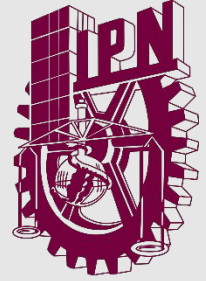


Doubly Linked List Deletion

Similar to singly linked list deletion, here we have three cases:

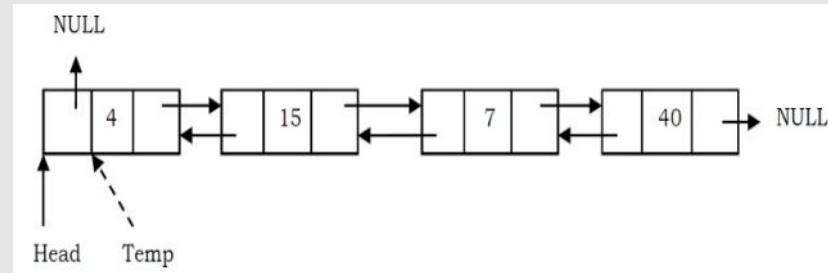
- Deleting the first node
- Deleting the last node
- Deleting an intermediate node

Deleting the First Node in Doubly Linked List

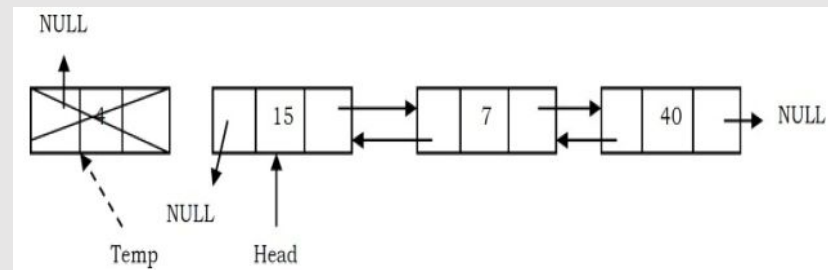


In this case, the first node (current head node) is removed from the list. It can be done in two steps:

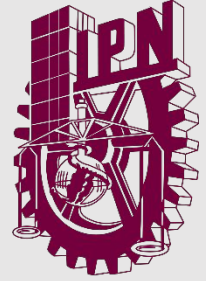
- Create a temporary node which will point to the same node as that of head.



- Now, move the head nodes pointer to the next node and change the head's left pointer to NULL. Then, dispose of the temporary node.

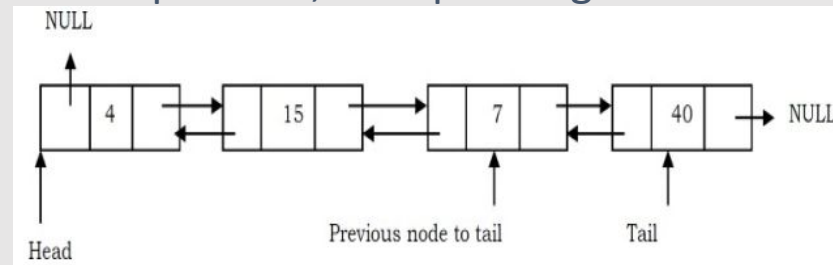


Deleting the Last Node in Doubly Linked List

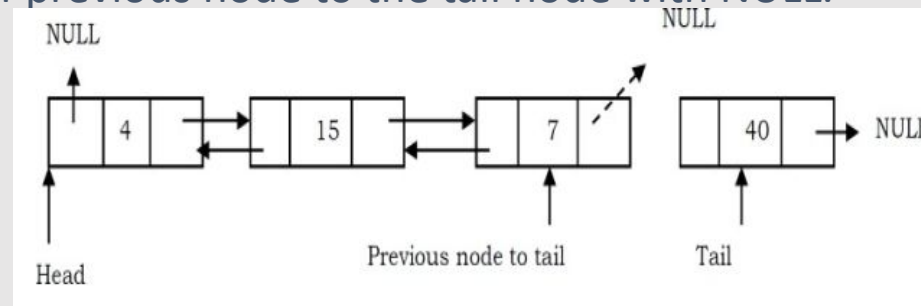


This operation is a bit trickier than removing the first node, because the algorithm should find a node, which is previous to the tail first. This can be done in three steps:

- Traverse the list and while traversing maintain the previous node address also. By the time we reach the end of the list, we will have two pointers, one pointing to the tail and the other pointing to the node before the tail.

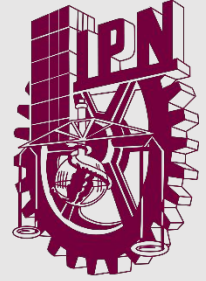


- Update the next pointer of previous node to the tail node with NULL.



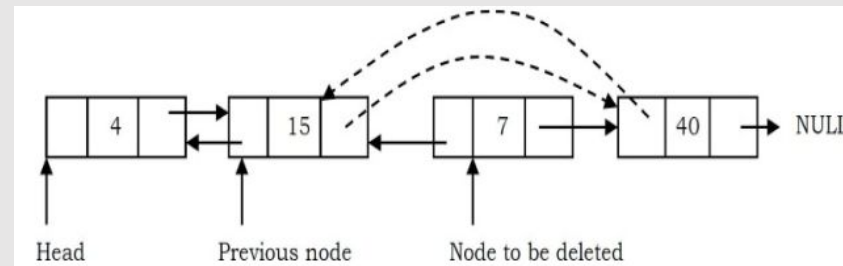
- Dispose the tail node.

Deleting an Intermediate Node in Doubly Linked List

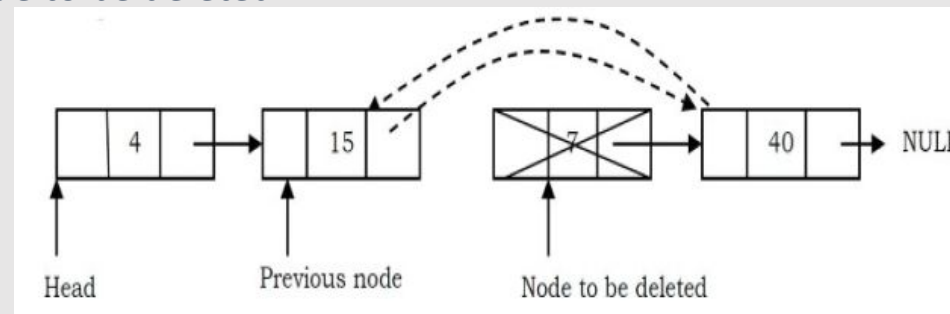


In this case, the node to be removed is *always located between* two nodes, and the head and tail links are not updated. The removal can be done in two steps:

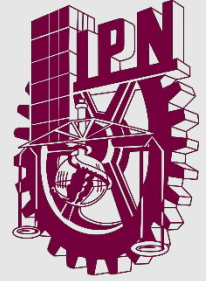
- Similar to the previous case, maintain the previous node while also traversing the list. Upon locating the node to be deleted, change the previous node's next pointer to the next node of the node to be deleted.



- Dispose of the current node to be deleted.

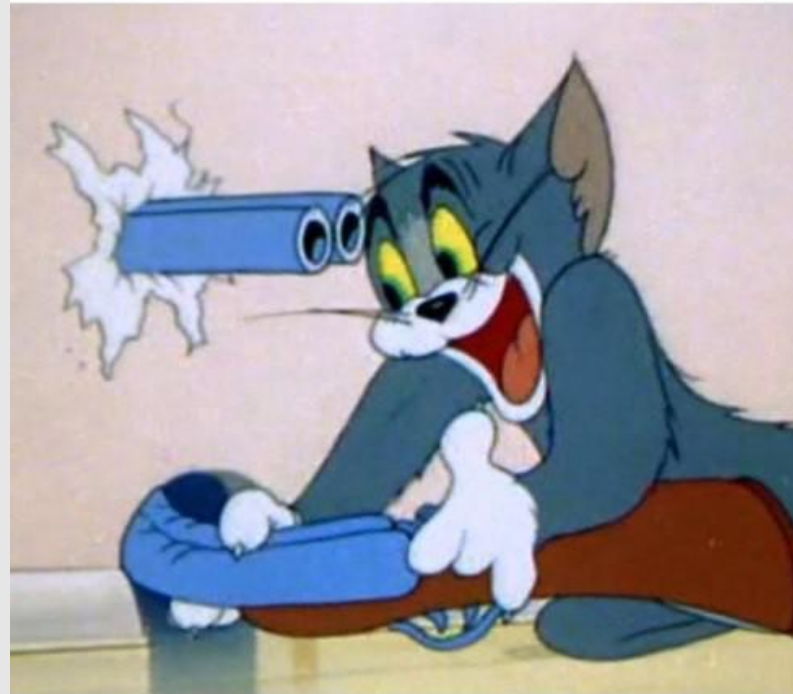


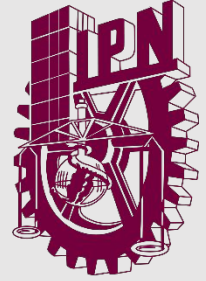
Circular Linked Lists



Absolutely no one :

Circular linked lists :





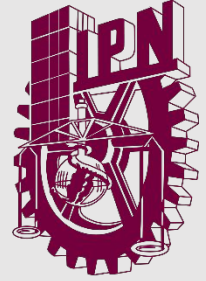
Circular linked list

- In singly linked lists and doubly linked lists, the end of lists are indicated with NULL value. But circular linked lists do not have ends. In circular linked lists, each node has a successor. Note that unlike singly linked lists, there is no node with NULL pointer in a circularly linked list.

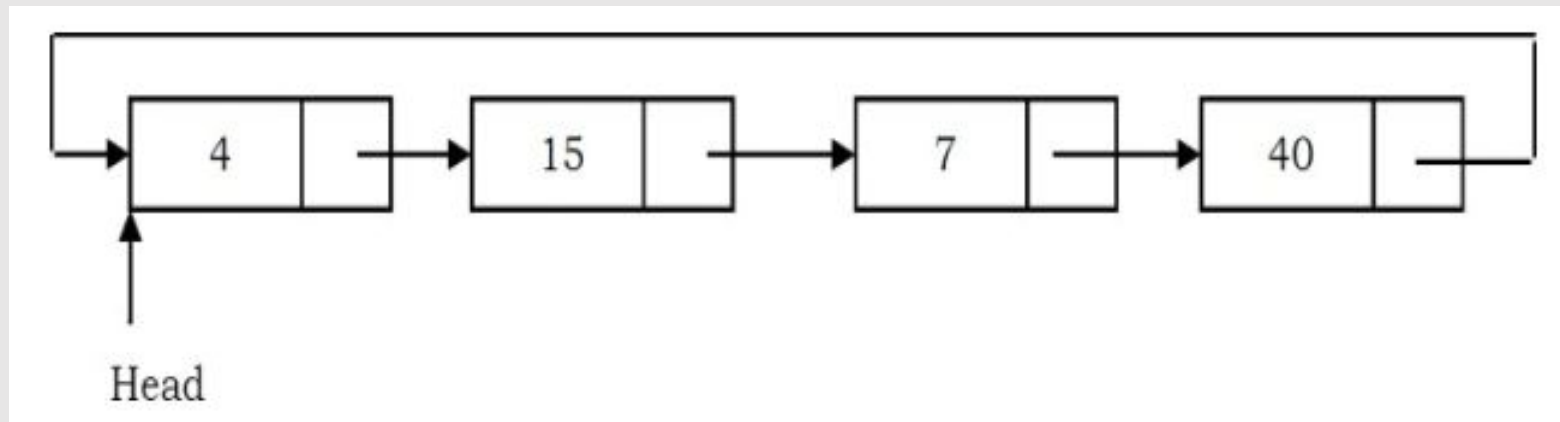
```
typedef struct CLLNode {  
    int data;  
    struct ListNode *next;  
};
```

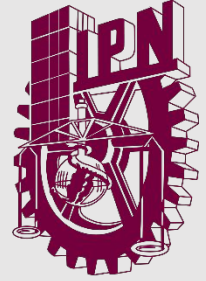
- In a circular linked list, we access the elements using the *head* node (similar to *head* node in singly linked list and doubly linked lists).

Counting Nodes in a Circular Linked List



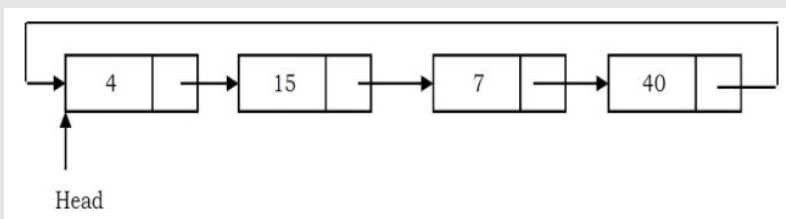
- The circular list is accessible through the node marked *head*. To count the nodes, the list has to be traversed from the node marked *head*, with the help of a dummy node *current*, and stop the counting when *current* reaches the starting node *head*.



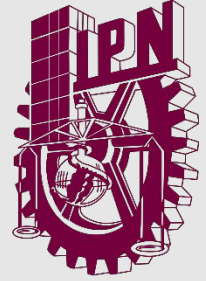


Counting Nodes in a Circular Linked List

- The circular list is accessible through the node marked *head*. To count the nodes, the list has to be traversed from the node marked *head*, with the help of a dummy node *current*, and stop the counting when *current* reaches the starting node *head*.

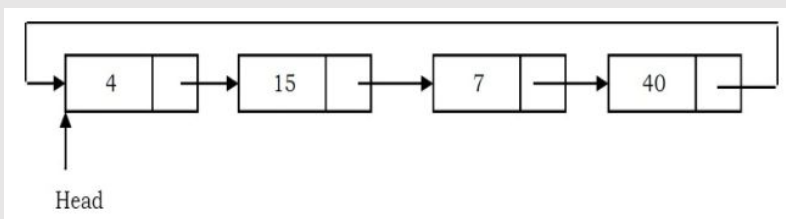


```
int CircularListLength(struct CLLNode *head) {  
    struct CLLNode *current = head;  
    int count = 0;  
    if(head == NULL)  
        return 0;  
  
    do {  
        current = current->next;  
        count++;  
    } while (current != head);  
  
    return count;  
}
```



Counting Nodes in a Circular Linked List

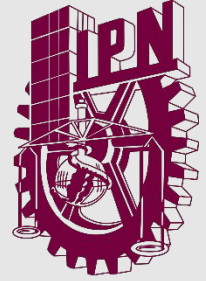
- The circular list is accessible through the node marked *head*. To count the nodes, the list has to be traversed from the node marked *head*, with the help of a dummy node *current*, and stop the counting when *current* reaches the starting node *head*.



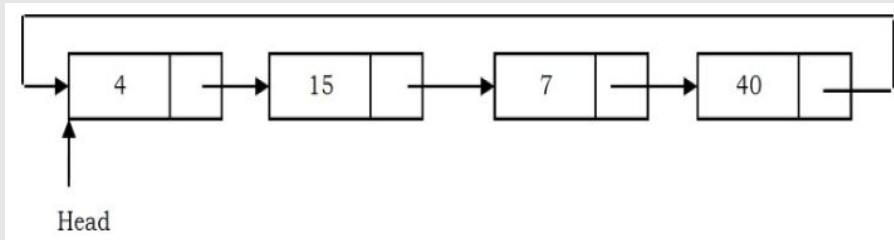
```
int CircularListLength(struct CLLNode *head) {  
    struct CLLNode *current = head;  
    int count = 0;  
    if(head == NULL)  
        return 0;  
  
    do {  
        current = current->next;  
        count++;  
    } while (current != head);  
  
    return count;  
}
```

Time Complexity: $O(n)$, for scanning the complete list of size n .

Printing the Contents of a Circular Linked List

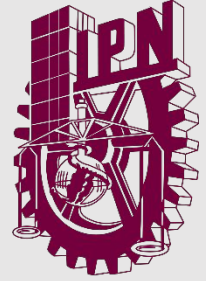


- Since all the nodes are arranged in a circular fashion, the *tail* node of the list will be the node previous to the *head* node.

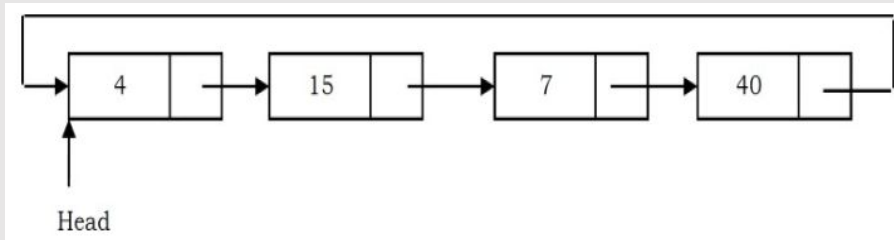


```
void PrintCircularListData(struct CLLNode *head) {  
    struct CLLNode *current = head;  
    if(head == NULL)  
        return;  
  
    do {  
        printf ("%d", current->data);  
        current = current->next;  
    } while (current != head);  
}
```

Printing the Contents of a Circular Linked List



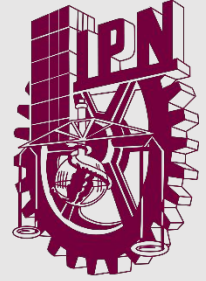
- Since all the nodes are arranged in a circular fashion, the *tail* node of the list will be the node previous to the *head* node.



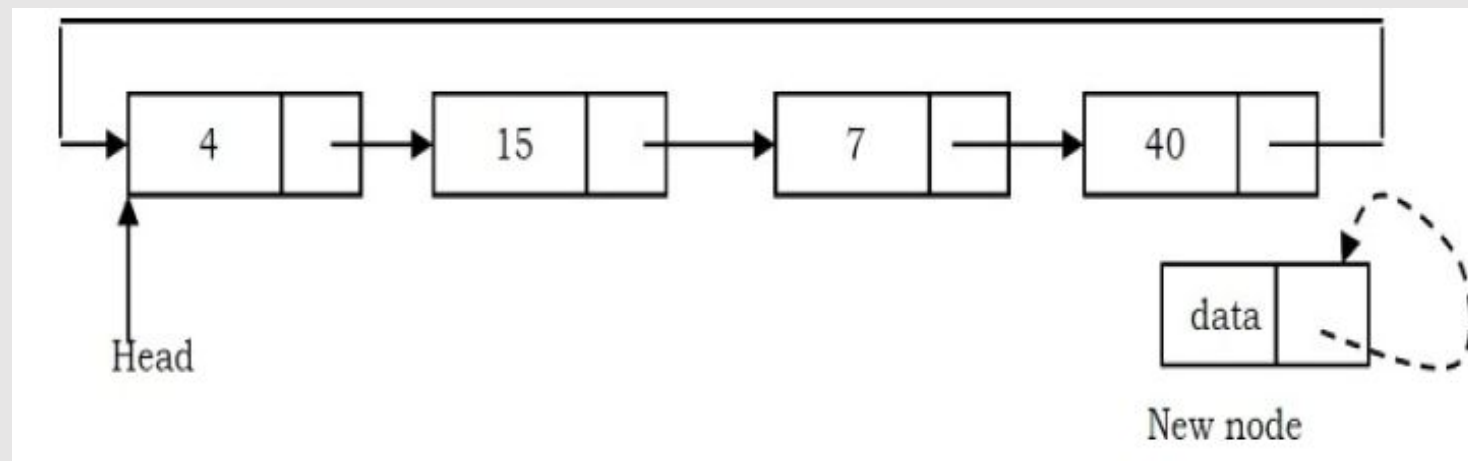
Time Complexity: $O(n)$, for scanning the complete list of size n .

```
void PrintCircularListData(struct CLLNode *head) {  
    struct CLLNode *current = head;  
    if(head == NULL)  
        return;  
  
    do {  
        printf ("%d", current->data);  
        current = current->next;  
    } while (current != head);  
}
```

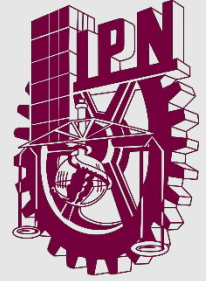
Inserting a Node at the End of a Circular Linked List



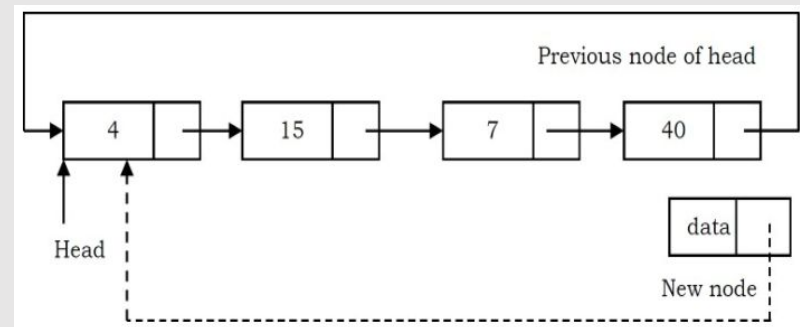
- The new node will be placed just after the tail node (which is the last node of the list), which means it will have to be inserted in between the tail node and the first node.
- Create a new node and initially keep its next pointer pointing to itself.



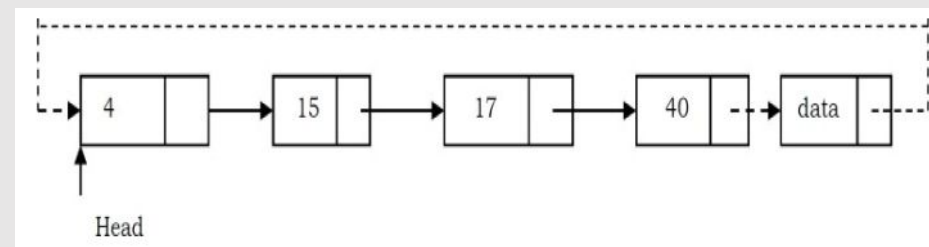
Inserting a Node at the End of a Circular Linked List



- Update the next pointer of the new node with the head node and also traverse the list to the tail. That means in a circular list we should stop at the node whose next node is head.



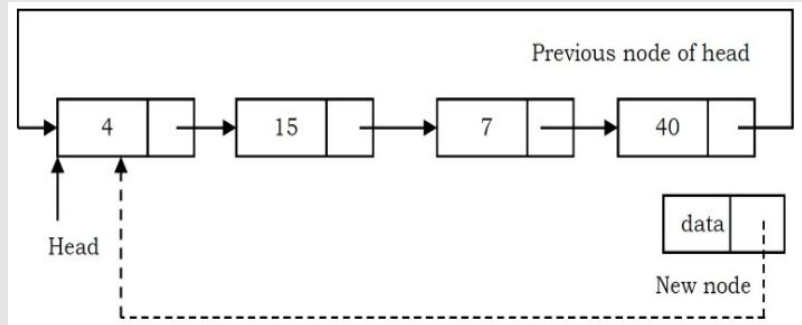
- Update the next pointer of the previous node to point to the new node and we get the list as shown below.



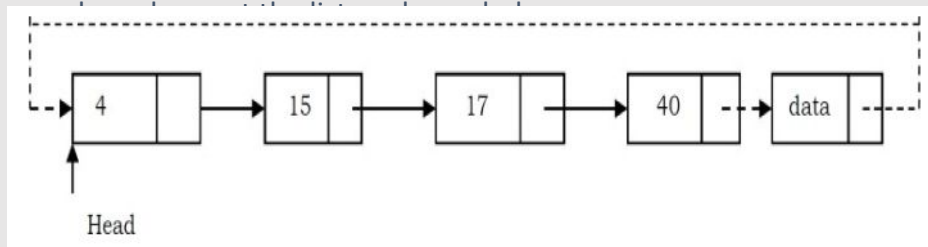
Inserting a Node at the End of a Circular Linked List



- Update the next pointer of the new node with the head node and also traverse the list to the tail. That means in a circular list we should stop at the node whose next node is head.



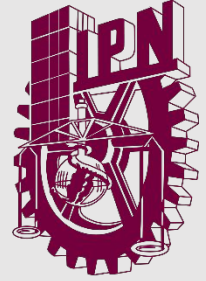
- Update the next pointer of the previous node to point to the new



Time Complexity: $O(n)$, for scanning the complete list of size n .

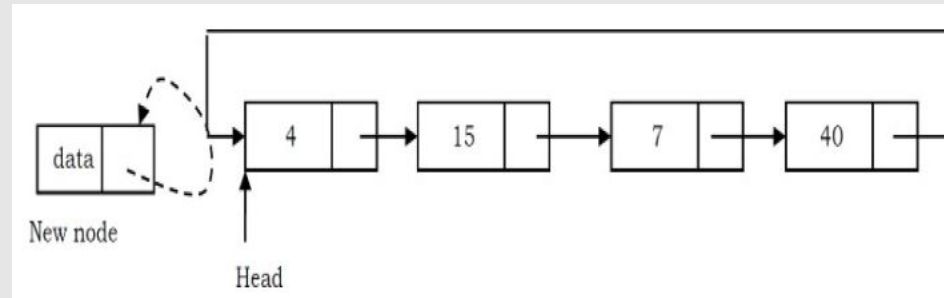
```
void InsertAtEndInCLL (struct CLLNode **head, int data) {  
    struct CLLNode *current = *head;  
    struct CLLNode *newNode = (struct CLLNode *) (malloc(sizeof(struct CLLNode)));  
    if(!newNode) {  
        printf("Memory Error");  
        return;  
    }  
    newNode->data = data;  
    while (current->next != *head)  
        current = current->next;  
  
    newNode->next = *head;  
  
    if(*head == NULL)  
        *head = newNode;  
    else {  
        newNode->next = *head;  
        current->next = newNode;  
    }  
}
```

Inserting a Node at the Front of a Circular Linked List

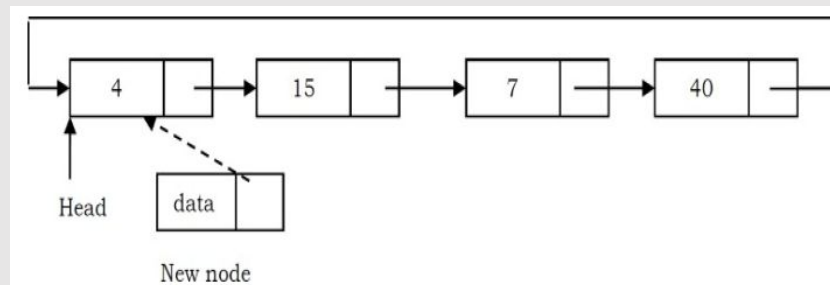


The only difference between inserting a node at the beginning and at the end is that, after inserting the new node, we just need to update the pointer. The steps for doing this are given below:

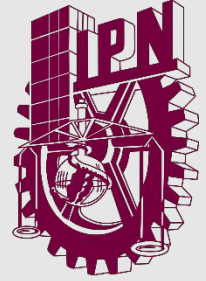
Create a new node and initially keep its next pointer pointing to itself.



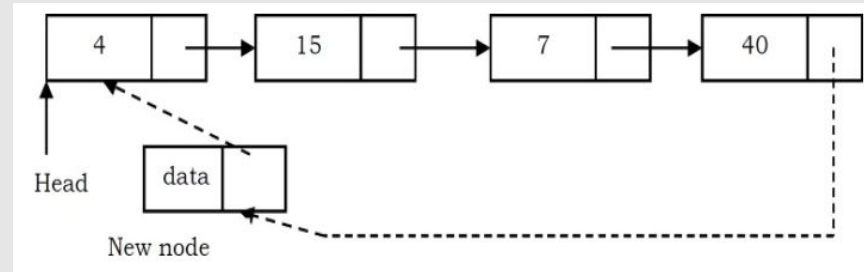
Update the next pointer of the new node with the head node and also traverse the list until the tail. That means in a circular list we should stop at the node which is its previous node in the list.



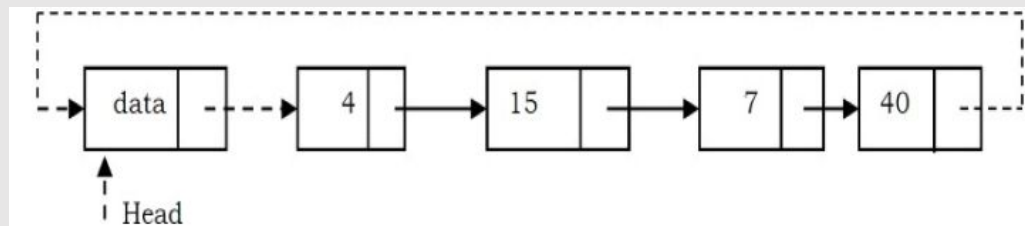
Inserting a Node at the Front of a Circular Linked List



Update the previous head node in the list to point to the new node.



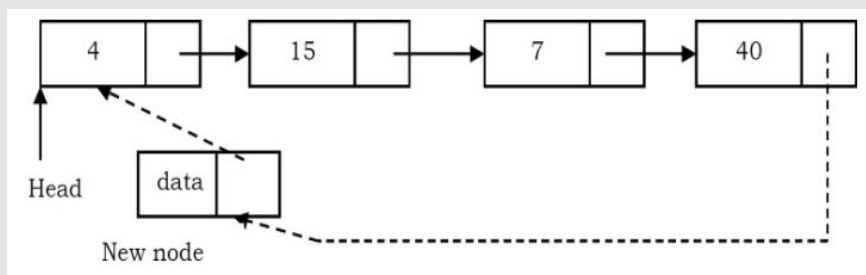
Make the new node as the head.



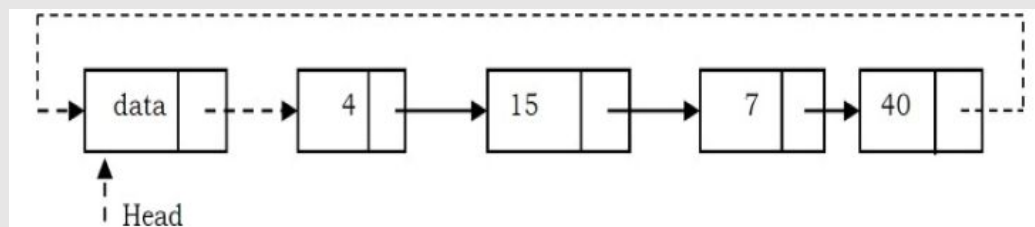


Inserting a Node at the Front of a Circular Linked List

Update the previous head node in the list to point to the new node.



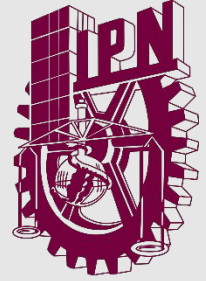
Make the new node as the head.



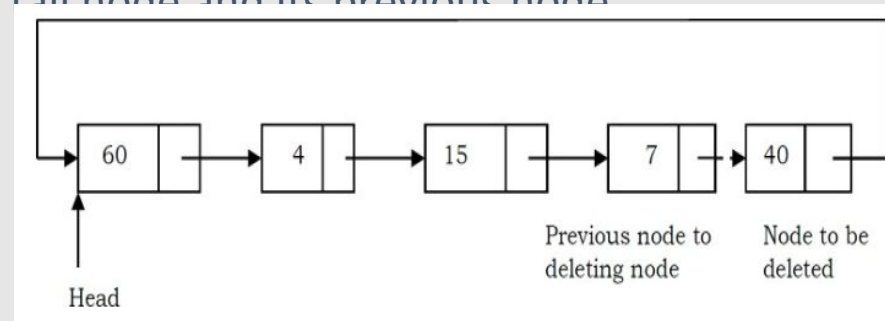
```
void InsertAtBeginInCLL (struct CLLNode **head, int data) {
    struct CLLNode *current = *head;
    struct CLLNode * newNode = (struct CLLNode *) (malloc(sizeof(struct CLLNode)));
    if(!newNode) {
        printf("Memory Error");
        return;
    }
    newNode->data = data;
    while (current->next != *head)
        current = current->next;
    newNode->next = *head;
    if(*head == NULL)
        *head = newNode;
    else {
        newNode->next = *head;
        current->next = newNode;
        *head = newNode;
    }
    return;
}
```

Time Complexity: $O(n)$, for scanning the complete list of size n

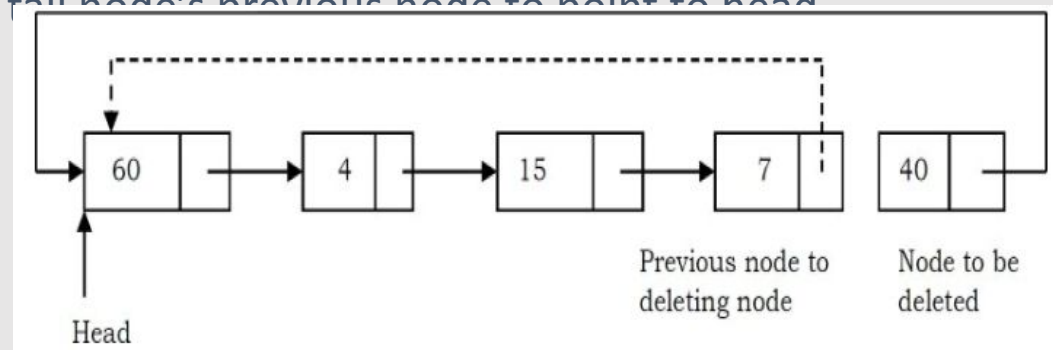
Deleting the Last Node in a Circular Linked List



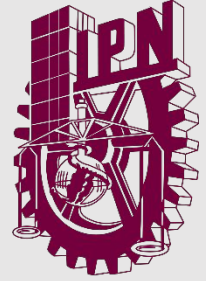
- To delete the last node 40, the list has to be traversed till you reach 7. The next field of 7 has to be changed to point to 60, and this node must be renamed *pTail*.
- Traverse the list and find the tail node and its previous node



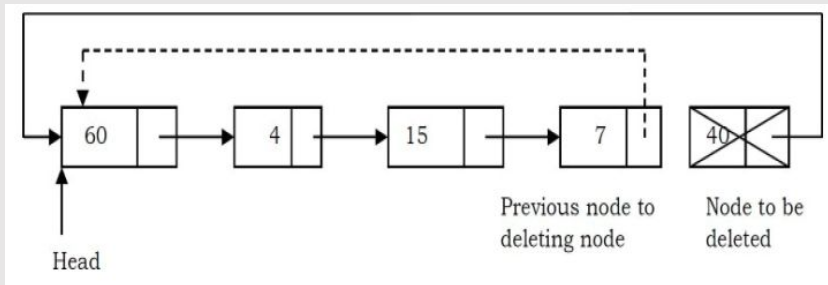
- Update the next pointer of tail node's previous node to point to head



Deleting the Last Node in a Circular Linked List



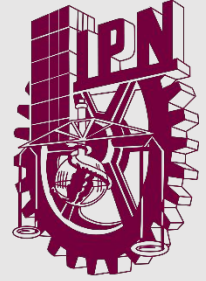
Dispose of the tail node.



Time Complexity: $O(n)$, for scanning the complete list of size n .

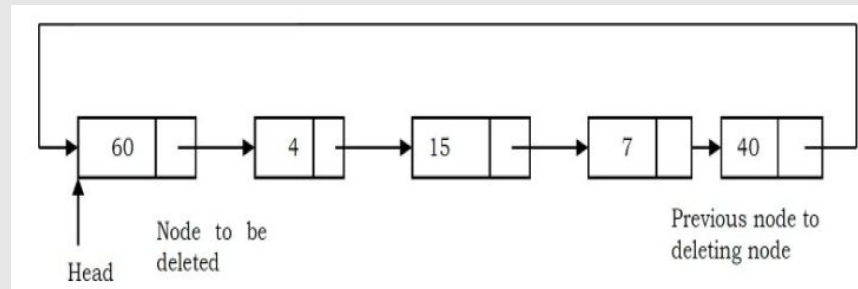
```
void DeleteLastNodeFromCLL (struct CLLNode **head) {
    struct CLLNode *temp = *head, *current = *head;
    if(*head == NULL) {
        printf( "List Empty"); return;
    }
    while (current->next != *head) {
        temp = current;
        current = current->next;
    }
    temp->next = current->next;
    free(current);
    return;
}
```

Deleting the First Node in a Circular List

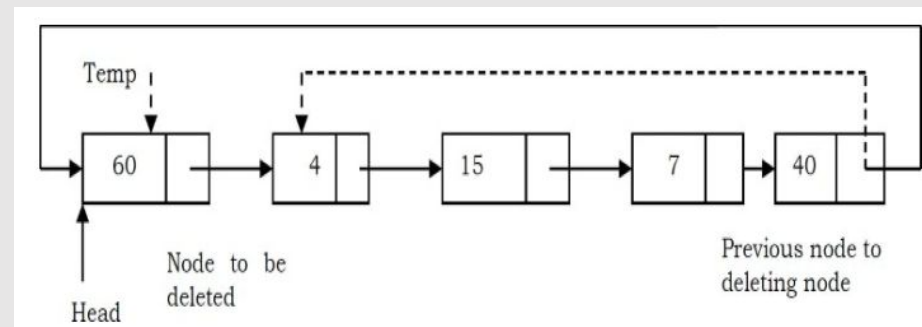


The first node can be deleted by simply replacing the next field of the tail node with the next field of the first node.

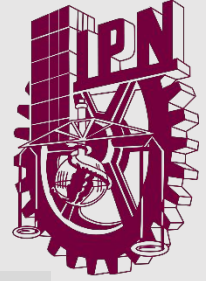
- Find the tail node of the linked list by traversing the list. Tail node is the previous node to the head node which we want to delete.



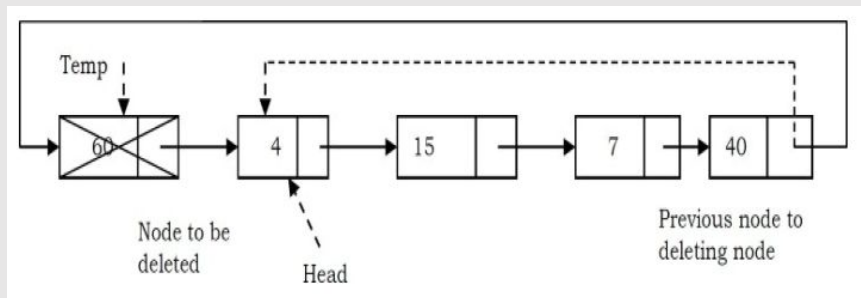
- Create a temporary node which will point to the head. Also, update the tail nodes next pointer to point to next node of head (as shown below).



Deleting the First Node in a Circular List



- Now, move the head pointer to next node. Create a temporary node which will point to head. Also, update the tail nodes next pointer to point to next node of head (as shown below).



Time Complexity: $O(n)$, for scanning the complete list of size n .

```
void DeleteFrontNodeFromCLL (struct CLLNode **head) {
    struct CLLNode *temp = *head;
    struct CLLNode *current = *head;

    if(*head == NULL) {
        printf("List Empty");
        return;
    }

    while (current->next != *head)
        current = current->next;

    current->next = *head->next;
    *head = *head->next;

    free(temp);
    return;
}
```